Administrator Guide

Browser Policy Manager 0.9.3

Contents

System requirements and delivery scope

- Minimum system requirements for BPM
- Assess current single-node source readiness
- Record production operating assumptions

Deploy BPM from source on Linux

- Prepare a Linux host for source deployment
- Set up the Linux source checkout
- Configure Linux source runtime settings
- Verify a Linux source deployment
- Install BPM from source on Ubuntu 26.04 LTS
- Install BPM from source on Debian 13.5
- Install BPM from source on Fedora Linux 44
- Install BPM from source on Linux Mint 22.3
- Install BPM from source on the Manjaro stable branch

Deploy BPM from source in Windows WSL

- Prepare Windows 10/11 for WSL source deployment
- Set up the BPM source checkout inside WSL
- Configure WSL runtime and browser access
- Verify BPM source deployment through WSL

Operate and update BPM from source

- Review DevOps configuration sources
- Plan storage, logs, and backup evidence
- Review network, CORS, and security limits
- Record source-operation boundaries
- Prepare evidence before updating from source
- Refresh the source revision and dependencies
- Run migrations and source-update checks
- Verify the update and apply rollback stop conditions
- Check BPM health and readiness

Integrate with the BPM API

- Integration audience and API boundary
- Supported integration patterns
- API conventions

- Current API limitations and security constraints
- Reuse DevOps API examples and environment templates
- Synchronize profile lifecycle data
- Archive, restore, permanently delete, or reset profiles
- Import Firefox policies.json through the API
- Export canonical Firefox policies.json through the API
- Validate candidate Firefox policies.json through the API

Run lifecycle workflows and recover from failures

- Run a pull, compare, and update control-product scenario
- Run an import, review, export, and compliance handoff scenario
- Pull and read BPM profiles from an external control product
- Validate and apply a profile update with expected_revision
- Run import, review, export, and compliance metadata handoff
- Gate control-product startup with BPM health and readiness
- Recover from integration failures and record audit evidence

Troubleshoot BPM and assess production readiness

- Troubleshoot failed startup and unreachable probes
- Troubleshoot schema cache and API validation errors
- Troubleshoot Firefox import and export failures
- Troubleshoot database and storage issues
- Troubleshoot WSL networking and stale dependencies
- Troubleshoot an unavailable documentation portal
- Review network exposure and proxy-readiness questions
- Plan monitoring inputs, backup evidence, and update windows

BPM documentation assistant

- Operate the documentation assistant in BPM 0.9.3
- Maintain the documentation-assistant boundary in BPM 0.9.3

System requirements and delivery scope

Minimum system requirements for BPM

Use these minimums for the supported BPM source workflow in release 0.9.3.

Base BPM

Run BPM through the supported Linux x86_64 source workflow: Ubuntu 26.04 LTS, Debian 13.5, Fedora Linux 44, Linux Mint 22.3, or the current Manjaro stable branch. Windows 10 or Windows 11 is supported only as a WSL 2 host for that Linux workflow. BPM does not provide a native Windows service or installer. Use CPython 3.14, two CPU cores, 4 GiB RAM, and 4 GB of writable free disk space for the checkout, virtual environment, database, logs and documentation build.

Browser, network and access

Use a current desktop browser with JavaScript and HTTPS enabled to operate the BPM web interface and documentation portal. The supported Firefox schema list is a policy-data compatibility statement, not a requirement to run the BPM UI in Firefox. Outbound HTTPS and sudo are required only while installing operating-system packages or source dependencies. The running process needs a normal Linux user that can write the BPM checkout and its configured data directories; do not run the application as root merely to operate it.

Documentation assistant in 0.9.3

The documentation-assistant chat is available as a reader interface, but release 0.9.3 does not deliver a local model, a RAG index, embeddings, generated answers, citations, or external-source access. It therefore adds no CPU, RAM, disk, GPU, NPU, cloud-account, or network requirement beyond the base BPM workflow.

Every submitted question shows the localized local-model training notice. Use the independent deterministic documentation search for current product information.

Assess current single-node source readiness

Decide whether the current source-run BPM instance is ready for controlled single-node operation and name the evidence that supports that decision.

Linux or Windows 10/11 through WSL source deployment is complete, dependencies are installed, and the intended `BPM_DATABASE_URL`, `BPM_DOCUMENTATION_SITE_DIR`, and `BPM_SCHEMA_CACHE_DIR` values are known.

BPM can be prepared for current-state single-node source operation. This assessment is not a production hardening certificate, an HA design, or a packaged service acceptance.

1. Record the exact source and runtime configuration.

   Capture `git rev-parse --short HEAD`, the BPM version, `BPM_HOST`, `BPM_PORT`, `BPM_DATABASE_URL`, `BPM_DOCUMENTATION_SITE_DIR`, `BPM_SCHEMA_CACHE_DIR`, and whether the process is started with `make dev` or a directly owned `uvicorn app.main:app` command.

2. Run liveness and readiness checks from the same network boundary users will use.

   ```
   export BPM_HOST="127.0.0.1"
   export BPM_PORT="8000"
   make dev
   curl -fsS http://127.0.0.1:8000/health
   curl -fsS http://127.0.0.1:8000/health/ready
   ```

   Record the result as `GET /health` and `GET /health/ready` evidence. Passing probes prove only source process liveness and readiness.

3. Check the data and documentation evidence before sharing the instance.

   Confirm that `data/bpm.db` is the intended database, that at least one profile can be exported as `policies.json`, and that `/help/` opens the built documentation artifact or a documented unavailable state.

4. Classify the instance as current-state source operation.

   > Warning:
   > Do not label the instance as packaged, hardened, load balanced, or highly available. If external controls add those properties, record them as external responsibilities and keep BPM evidence limited to single-node source operation.

The readiness note says what can be safely done today from source, which checks passed, and which decisions must wait for distribution and runtime support.

Use the network exposure and proxy-readiness topic before exposing the instance beyond a local administrator workstation or controlled lab host.

Record production operating assumptions

Record the production, HA, reverse-proxy, restore, and service-management responsibilities for a source-run instance.

Readiness, network exposure, backup/export, monitoring, and update-window notes exist for the current source-run instance.

This topic records the operating assumptions for a source-run instance and the platform responsibilities that must be assigned before shared use.

1. Name the supported current shape.

   Write that BPM Administrator/DevOps documentation supports source deployment, manual configuration, health/readiness checks, API examples, manual backup evidence, documentation rebuild checks, and single-node update windows.

2. Record service and proxy responsibilities.

   Assign owners for Linux or Windows services, reverse proxy, TLS, secrets, authentication gateway configuration, and production hardening.

3. Record HA and upgrade design requirements.

   Capture requirements for multi-node coordination, load balancing, shared storage, migrations, failover, restore, and upgrade windows.

4. Capture operating inputs.

   > Warning:
   > Do not treat multiple source nodes behind a load balancer as an operating design without documented coordination and recovery procedures. Capture RTO/RPO, backup verification, TLS ownership, secrets ownership, logging, monitoring, and update-window expectations.

The operating record captures the responsibilities and recovery expectations for shared use.

Use this record during operational review and update planning.

Deploy BPM from source on Linux

Prepare a Linux host for source deployment

Confirm host prerequisites and operating responsibilities before installing BPM from a repository checkout.

Use a Linux host where you can install Python 3.14+, Git, build tools, and curl. This runbook assumes a source checkout, not a packaged installer.

This guide describes a BPM deployment from a source checkout. Keep the checkout, runtime, data, and organization-owned platform controls under a controlled operating procedure.

1. Identify the deployment purpose and record that this is a source-based Linux deployment.

   Expected result: the operations note identifies the Git checkout, runtime owner, and operating procedure.

2. Install or verify host prerequisites.

   Required tools are Git, Python 3.14+, pip, make, curl, and a shell that can run `source .venv/bin/activate`. The repository commands use the local virtual environment and do not require a global BPM package.

3. Create a working directory that is separate from production data and secrets.

   Keep the repository checkout, `.venv`, and local SQLite data under a controlled path. Do not store real secrets in documentation examples or committed files.

4. Record external platform responsibilities before continuing.

   > Warning:
   > Document the organization-owned procedures for services, reverse proxy and TLS, secrets, backups, restore, monitoring, and production hardening before exposing the instance to users.

The host and operating assumptions are ready for a supported BPM source checkout workflow.

Continue with the Linux source checkout topic before configuring runtime variables.

Set up the Linux source checkout

Clone the BPM repository, create the Python virtual environment, install development dependencies, and prepare the local database schema.

The Linux host is prepared, and the administrator has access to the BPM repository URL and the approved target branch, tag, or revision.

The repository README defines the supported source setup sequence. Use repository-local commands so later troubleshooting can compare the environment with committed scripts.

1. Clone the repository and enter the checkout.

   ```
   git clone <repository-url> browser-policy-manager
   cd browser-policy-manager
   git checkout <approved-0.9.1-ref>
   ```

   Replace placeholders with the approved internal repository URL and release reference.

2. Create and activate the local Python environment.

   ```
   python -m venv .venv
   source .venv/bin/activate
   python -m pip install --upgrade pip
   ```

   The virtual environment stays inside the checkout and is not committed.

3. Install BPM source dependencies from the checkout.

   ```
   pip install -e ".[dev]"
   ```

   The editable install keeps runtime code and tests aligned with the current source tree.

4. Apply database migrations for the selected runtime database.

   ```
   alembic upgrade head
   ```

   > Warning:
   > Run migrations only against the intended BPM database. For the default source workflow this is the local SQLite database, not a production service database.

The checkout has an activated Python environment, installed dependencies, and a migrated database schema.

Configure the runtime environment before starting the application process.

Configure Linux source runtime settings

Set the database URL, host, port, reload mode, CORS, and documentation-site path used by the source-run BPM process.

The checkout dependencies are installed and migrations have been applied for the database you plan to use.

BPM reads environment variables with the `BPM_` prefix and also supports a repository-local `.env` file. The default database URL is `sqlite+aiosqlite:///./data/bpm.db`, which stores data in `data/bpm.db`.

1. Create the local data directory and define the database URL.

   ```
   mkdir -p data
   export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
   ```

   For repeatable source runs, the same value may be stored in a local `.env` file that is not committed.

2. Set network and reload values for the local source process.

   ```
   export BPM_HOST="127.0.0.1"
   export BPM_PORT="8000"
   export BPM_RELOAD="true"
   ```

   Use loopback for local verification. Binding to a shared interface is an operational decision outside this source-runbook boundary.

3. Review CORS and documentation-site settings.

   The source defaults include `BPM_ENABLE_CORS`, `BPM_CORS_ALLOW_ORIGINS`, and `BPM_DOCUMENTATION_SITE_DIR`. Keep permissive CORS only for controlled local evaluation, and point `BPM_DOCUMENTATION_SITE_DIR` at an installed static documentation site when one is available.

4. Start BPM from the repository script.

   ```
   make dev
   ```

   The Makefile runs `uvicorn app.main:app --reload --port 8000`. Keep the foreground terminal open so startup messages and tracebacks are visible.

BPM is running from source with explicit local runtime settings.

Verify health, readiness, the product UI, and first administrative smoke checks.

Verify a Linux source deployment

Use health probes, the web UI, and logs to confirm that the source-run BPM instance is usable.

The BPM process is running from the repository checkout with the intended `BPM_DATABASE_URL`, host, port, and documentation-site settings.

The current application exposes liveness at `/health` and readiness at `/health/ready`. These probes confirm the running process response only; they do not prove production hardening, Firefox enforcement, backup readiness, or HA behavior.

1. Check liveness and readiness from the Linux host.

   ```
   curl -fsS http://127.0.0.1:8000/health
   curl -fsS http://127.0.0.1:8000/health/ready
   ```

   Expected responses are `{"status":"ok"}` and `{"status":"ready","ready":true}`.

2. Open the product UI and confirm that the Library route responds.

   Use `http://127.0.0.1:8000/profiles`. The route should load the BPM interface or show an intentional application state, not a server traceback.

3. Inspect the foreground application logs.

   Review the terminal running `make dev` or `uvicorn app.main:app --reload --port 8000`. Record startup errors, migration errors, port conflicts, and documentation-site warnings without copying secrets into tickets.

4. Record the support boundary for the verification result.

   > Warning:
   > Passing health probes, the UI route, and foreground-log checks does not certify production readiness, reverse proxy behavior, managed backups, or Firefox runtime policy enforcement.

The administrator has evidence that the Linux source deployment starts, answers health/readiness probes, serves the UI route, and has no immediate foreground-log blockers.

If verification fails, keep the command, environment variable names, probe responses, and relevant log lines together for troubleshooting. Do not broaden this runbook into deferred installer or production-topology instructions.

Install BPM from source on Ubuntu 26.04 LTS

Run the complete BPM 0.9.1 source-install sequence on Ubuntu 26.04 LTS with the distribution Python 3.14 packages.

Use Ubuntu 26.04 LTS with sudo access, outbound HTTPS, and an approved BPM 0.9.1 Git ref. Replace only `<approved-0.9.1-ref>` before running the commands.

This procedure creates a single-node source-run instance for controlled evaluation. It does not install a system service or provide production hardening.

1. Verify Ubuntu and install prerequisites.

   ```
   . /etc/os-release
   test "$ID" = "ubuntu" && test "$VERSION_ID" = "26.04"
   sudo apt-get update
   sudo apt-get install -y build-essential ca-certificates curl git make python3.14 python3.14-dev
   python3.14-venv
   export BPM_PYTHON=/usr/bin/python3.14
   "$BPM_PYTHON" --version
   ```

   Stop unless the final command reports Python 3.14.x.

2. Check out the approved BPM source ref.

   ```
   export BPM_REPOSITORY_URL="https://github.com/Goudron/browser-policy-manager.git"
   export BPM_REF="<approved-0.9.1-ref>"
   test "$BPM_REF" != "<approved-0.9.1-ref>"
   cd "$HOME"
   git clone "$BPM_REPOSITORY_URL" browser-policy-manager
   cd "$HOME/browser-policy-manager"
   git checkout --detach "$BPM_REF"
   git status --short
   git rev-parse --short HEAD
   ```

   Stop if the ref is not approved or the new checkout is not clean.

3. Create the environment, install BPM, and migrate the new database.

   ```
   "$BPM_PYTHON" -m venv .venv
   source .venv/bin/activate
   python -m pip install --upgrade pip
   pip install -e ".[dev]"
   python --version
   mkdir -p data
   export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
   alembic upgrade head
   ```

4. Set the local runtime and start BPM in the foreground.

```
export BPM_HOST="127.0.0.1"
export BPM_PORT="8000"
export BPM_RELOAD="true"
make dev
```

Keep this terminal open. Use a second terminal for verification.

5. Verify probes, UI, and logs from a second terminal.

```
cd browser-policy-manager
source .venv/bin/activate
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
curl -fsS -o /dev/null http://127.0.0.1:8000/profiles
```

Expect `{"status":"ok"}`, `{"status":"ready","ready":true}`, and a successful UI response. Review the foreground terminal for tracebacks.

6. Apply the fresh-install stop and rollback boundary.

If any check fails, press `Ctrl+C`, preserve the command output and foreground logs, and do not edit `data/bpm.db`. Because this is a new installation, rollback means retaining or renaming the failed checkout for diagnosis; it does not mean running `alembic downgrade`.

The Ubuntu 26.04 LTS host has a migrated BPM 0.9.1 source checkout, passing probes, a responding Library, and reviewable foreground logs.

The command sequence passed in a clean Ubuntu 26.04 userspace container. This evidence does not verify native boot, kernel, systemd, desktop, hardware, host firewall, or production deployment.

Install BPM from source on Debian 13.5

Run the complete BPM 0.9.1 source-install sequence on Debian 13.5 by installing a checksum-verified CPython 3.14.6 under the current user.

Use Debian 13.5 (trixie) with sudo access, outbound HTTPS, at least 4 GB free build space, and an approved BPM 0.9.1 Git ref.

Debian 13 defaults to Python 3.13, below BPM's Python 3.14 requirement. This procedure does not replace the system Python; it installs CPython 3.14.6 under `$HOME/.local`.

1. Verify Debian and install CPython build prerequisites.

```
. /etc/os-release
test "$ID" = "debian" && test "$VERSION_ID" = "13"
sudo apt-get update
sudo apt-get install -y build-essential ca-certificates curl git make libbz2-dev libffi-dev libgdbm-dev liblzma-dev libncurses-dev libreadline-dev libsqlite3-dev libssl-dev tk-dev uuid-dev xz-utils zlib1g-dev
```

2. Build the verified CPython 3.14.6 source release.

```
export PYTHON_VERSION="3.14.6"
export PYTHON_SHA256="143b1dddefaec3bd2e21e3b839b34a2b7fb9842272883c576420d605e9f30c63"
export PYTHON_PREFIX="$HOME/.local/python-$PYTHON_VERSION"
mkdir -p "$HOME/src" && cd "$HOME/src"
curl -fLO "https://www.python.org/ftp/python/$PYTHON_VERSION/Python-$PYTHON_VERSION.tar.xz"
printf '%s  %s\n' "$PYTHON_SHA256" "Python-$PYTHON_VERSION.tar.xz" | sha256sum -c -
tar -xf "Python-$PYTHON_VERSION.tar.xz"
cd "Python-$PYTHON_VERSION"
./configure --prefix="$PYTHON_PREFIX" --with-ensurepip=install
make -j"$(nproc)"
make altinstall
export BPM_PYTHON="$PYTHON_PREFIX/bin/python3.14"
"$BPM_PYTHON" --version
```

   Stop on checksum, configure, build, or version failure.

3. Check out the approved BPM source ref.

```
cd "$HOME"
export BPM_REPOSITORY_URL="https://github.com/Goudron/browser-policy-manager.git"
export BPM_REF="<approved-0.9.1-ref>"
test "$BPM_REF" != "<approved-0.9.1-ref>"
git clone "$BPM_REPOSITORY_URL" browser-policy-manager
cd browser-policy-manager
git checkout --detach "$BPM_REF"
git status --short
git rev-parse --short HEAD
```

4. Create the environment, install BPM, and migrate the new database.

```
"$BPM_PYTHON" -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"
python --version
mkdir -p data
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
alembic upgrade head
```

5. Start BPM in the foreground.

   ```
   export BPM_HOST="127.0.0.1"
   export BPM_PORT="8000"
   export BPM_RELOAD="true"
   make dev
   ```

   Keep this terminal open for logs.

6. Verify probes and the UI from a second terminal.

   ```
   cd "$HOME/browser-policy-manager"
   source .venv/bin/activate
   curl -fsS http://127.0.0.1:8000/health
   curl -fsS http://127.0.0.1:8000/health/ready
   curl -fsS -o /dev/null http://127.0.0.1:8000/profiles
   ```

7. Apply the stop and rollback boundary.

   On failure, press `Ctrl+C`, preserve logs and checksum output, and do not run `alembic downgrade`. Keep the user-owned Python prefix and failed checkout until diagnosis; remove them only after confirming they contain no required data.

The Debian 13.5 host has a user-owned Python 3.14.6 runtime, migrated BPM source checkout, passing probes, responding UI, and foreground logs.

The command sequence passed in a clean Debian 13.5 userspace container. CPython reported the optional `_dbm` and `_zstd` modules as unavailable; the complete BPM workflow still passed. This evidence does not verify native boot, kernel, systemd, desktop, hardware, host firewall, or production deployment.

Install BPM from source on Fedora Linux 44

Run the complete BPM 0.9.1 source-install sequence on Fedora Linux 44 with its distribution Python 3.14 runtime.

Use Fedora Linux 44 with sudo access, outbound HTTPS, and an approved BPM 0.9.1 Git ref.

This is a foreground single-node source workflow, not a systemd or production deployment recipe.

1. Verify Fedora and install prerequisites.

   ```
   . /etc/os-release
   test "$ID" = "fedora" && test "$VERSION_ID" = "44"
   sudo dnf install -y ca-certificates curl gcc gcc-c++ git make python3 python3-devel
   export BPM_PYTHON=/usr/bin/python3
   "$BPM_PYTHON" --version
   ```

   Stop unless Python 3.14.x is reported.

2. Check out the approved BPM source ref.

   ```
   export BPM_REPOSITORY_URL="https://github.com/Goudron/browser-policy-manager.git"
   export BPM_REF="<approved-0.9.1-ref>"
   test "$BPM_REF" != "<approved-0.9.1-ref>"
   cd "$HOME"
   git clone "$BPM_REPOSITORY_URL" browser-policy-manager
   cd "$HOME/browser-policy-manager"
   git checkout --detach "$BPM_REF"
   git status --short
   git rev-parse --short HEAD
   ```

3. Create the environment, install BPM, and migrate.

   ```
   "$BPM_PYTHON" -m venv .venv
   source .venv/bin/activate
   python -m pip install --upgrade pip
   pip install -e ".[dev]"
   python --version
   mkdir -p data
   export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
   alembic upgrade head
   ```

4. Start BPM in the foreground.

   ```
   export BPM_HOST="127.0.0.1"
   export BPM_PORT="8000"
   export BPM_RELOAD="true"
   make dev
   ```

Use a second terminal for checks.

5. Verify probes, UI, and foreground logs.

```
cd browser-policy-manager
source .venv/bin/activate
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
curl -fsS -o /dev/null http://127.0.0.1:8000/profiles
```

6. Apply the stop boundary.

   On failure, press `Ctrl+C`, preserve DNF, pip, migration, build, probe, and log evidence, and do not run `alembic downgrade`.

The Fedora Linux 44 host has a migrated BPM source checkout, passing probes, responding UI, and foreground logs.

The command sequence passed in a clean Fedora 44 userspace container. This evidence does not verify native boot, kernel, systemd, desktop, hardware, host firewall, or production deployment.

Install BPM from source on Linux Mint 22.3

Run the complete BPM 0.9.1 source-install sequence on Linux Mint 22.3 by installing a checksum-verified CPython 3.14.6 under the current user.

Use Linux Mint 22.3 (Zena) with sudo access, outbound HTTPS, at least 4 GB free build space, and an approved BPM 0.9.1 Git ref.

Linux Mint 22.3 uses the Ubuntu Noble package base, whose default Python is below BPM's 3.14 requirement. The system Python is not replaced.

1. Verify Mint and install CPython build prerequisites.

```
. /etc/os-release
test "$ID" = "linuxmint" && test "$VERSION_ID" = "22.3"
sudo apt-get update
sudo apt-get install -y build-essential ca-certificates curl git make libbz2-dev libffi-dev libgdbm-dev liblzma-dev libncurses-dev libreadline-dev libsqlite3-dev libssl-dev tk-dev uuid-dev xz-utils zlib1g-dev
```

2. Build the verified CPython 3.14.6 source release.

```
export PYTHON_VERSION="3.14.6"
export PYTHON_SHA256="143b1dddefaec3bd2e21e3b839b34a2b7fb9842272883c576420d605e9f30c63"
export PYTHON_PREFIX="$HOME/.local/python-$PYTHON_VERSION"
mkdir -p "$HOME/src" && cd "$HOME/src"
curl -fLO "https://www.python.org/ftp/python/$PYTHON_VERSION/Python-$PYTHON_VERSION.tar.xz"
printf '%s  %s\n' "$PYTHON_SHA256" "Python-$PYTHON_VERSION.tar.xz" | sha256sum -c -
tar -xf "Python-$PYTHON_VERSION.tar.xz"
cd "Python-$PYTHON_VERSION"
./configure --prefix="$PYTHON_PREFIX" --with-ensurepip=install
make -j"$(nproc)"
make altinstall
export BPM_PYTHON="$PYTHON_PREFIX/bin/python3.14"
"$BPM_PYTHON" --version
```

3. Check out the approved BPM source ref.

```
cd "$HOME"
export BPM_REPOSITORY_URL="https://github.com/Goudron/browser-policy-manager.git"
export BPM_REF="<approved-0.9.1-ref>"
test "$BPM_REF" != "<approved-0.9.1-ref>"
git clone "$BPM_REPOSITORY_URL" browser-policy-manager
cd browser-policy-manager
git checkout --detach "$BPM_REF"
git status --short
git rev-parse --short HEAD
```

4. Create the environment, install BPM, and migrate.

```
"$BPM_PYTHON" -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"
python --version
mkdir -p data
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
alembic upgrade head
```

5. Start BPM in the foreground.

```
export BPM_HOST="127.0.0.1"
export BPM_PORT="8000"
export BPM_RELOAD="true"
make dev
```

6. Verify probes and the UI from a second terminal.

```
cd "$HOME/browser-policy-manager"
source .venv/bin/activate
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
curl -fsS -o /dev/null http://127.0.0.1:8000/profiles
```

7. Apply the stop and rollback boundary.

   On failure, press `Ctrl+C`, preserve logs and checksum output, and do not run `alembic downgrade`. Keep the user-owned Python prefix and checkout until diagnosis.

The Linux Mint 22.3 host has a user-owned Python 3.14.6 runtime, migrated BPM source checkout, passing probes, responding UI, and foreground logs.

The command sequence passed in a clean Linux Mint 22.3 userspace container. CPython reported the optional `_dbm` and `_zstd` modules as unavailable; the complete BPM workflow still passed. This evidence does not verify native boot, kernel, systemd, desktop, hardware, host firewall, or production deployment.

Install BPM from source on the Manjaro stable branch

Run the complete BPM 0.9.1 source-install sequence after fully updating the Manjaro stable branch and verifying Python 3.14.

Use the Manjaro stable branch at or after the 2026-06-26 update with sudo access, outbound HTTPS, and an approved BPM 0.9.1 Git ref.

Manjaro is rolling-release. The transcript must record the branch and fully updated package state; a desktop image version alone is not sufficient evidence.

1. Verify the stable branch, update the system, and install prerequisites.

   ```
   . /etc/os-release
   test "$ID" = "manjaro"
   test "$(pacman-mirrors -G)" = "stable"
   sudo pacman -Syu --needed --noconfirm base-devel ca-certificates curl git python python-pip
   pacman-mirrors -G
   uname -r
   export BPM_PYTHON=/usr/bin/python
   "$BPM_PYTHON" --version
   ```

   Stop unless the branch is stable, the full update succeeds, and Python 3.14.x is reported. Do not perform a partial upgrade.

2. Check out the approved BPM source ref.

   ```
   export BPM_REPOSITORY_URL="https://github.com/Goudron/browser-policy-manager.git"
   export BPM_REF="<approved-0.9.1-ref>"
   test "$BPM_REF" != "<approved-0.9.1-ref>"
   cd "$HOME"
   git clone "$BPM_REPOSITORY_URL" browser-policy-manager
   cd "$HOME/browser-policy-manager"
   git checkout --detach "$BPM_REF"
   git status --short
   git rev-parse --short HEAD
   ```

3. Create the environment, install BPM, and migrate.

   ```
   "$BPM_PYTHON" -m venv .venv
   source .venv/bin/activate
   python -m pip install --upgrade pip
   pip install -e ".[dev]"
   python --version
   mkdir -p data
   export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
   alembic upgrade head
   ```

4. Start BPM in the foreground.

```
export BPM_HOST="127.0.0.1"
export BPM_PORT="8000"
export BPM_RELOAD="true"
make dev
```

5. Verify probes, UI, and logs from a second terminal.

```
cd browser-policy-manager
source .venv/bin/activate
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
curl -fsS -o /dev/null http://127.0.0.1:8000/profiles
```

6. Apply the stop boundary.

   On failure, press `Ctrl+C`, preserve branch, package, migration, build, probe, and log evidence, and do not downgrade individual Manjaro packages or run `alembic downgrade` as an ad hoc rollback.

The fully updated Manjaro stable host has a migrated BPM source checkout, passing probes, responding UI, and foreground logs.

The command sequence passed in a clean Manjaro stable userspace container after recording the branch and full package state. This evidence does not verify native boot, kernel, systemd, desktop, hardware, host firewall, or production deployment.

Deploy BPM from source in Windows WSL

Prepare Windows 10/11 for WSL source deployment

Enable WSL, choose a supported Linux distribution assumption, and confirm that BPM will run inside Linux rather than as a native Windows service.

Use Windows 10 or Windows 11 with permission to enable WSL 2, install a Linux distribution such as Ubuntu LTS, and run commands in both PowerShell and the WSL shell.

This runbook documents BPM source deployment through WSL. Run the application through the Linux source workflow inside WSL and record any Windows platform controls owned by your organization.

1. Confirm WSL availability from PowerShell.

   ```
   wsl --status
   wsl --list --verbose
   ```

   Expected result: WSL is installed, WSL 2 is available, and the chosen Linux distribution is visible.

2. Install WSL and an approved Linux distribution if they are missing.

   ```
   wsl --install
   wsl --install -d Ubuntu
   ```

   Use your organization-approved distribution name. Reboot if Windows requests it, then create the Linux user when the distribution first starts.

3. Record the distribution and filesystem boundary.

   Place BPM under the WSL Linux filesystem, for example `/home/<linux-user>/browser-policy-manager`. Avoid running the checkout from `/mnt/c` because Windows filesystem semantics and antivirus scanning can make Python environments, file watching, and permissions unreliable.

4. Record the WSL operating boundary.

   > Warning:
   > This is a Linux source workflow inside WSL. Document the organization-owned procedures for Windows service management, Event Viewer, firewall policy, reverse proxy and TLS, backups, secrets, and production hardening.

Windows and WSL are ready for a Linux-based BPM source deployment with clear platform and filesystem boundaries.

Continue by opening the WSL distribution shell and setting up the source checkout under the Linux home directory.

Set up the BPM source checkout inside WSL

Install Linux prerequisites in WSL, clone BPM into the Linux filesystem, create the virtual environment, install dependencies, and apply migrations.

A WSL 2 Linux distribution is installed and opened, and the administrator has the approved BPM repository URL and 0.9.1 branch or tag.

All commands in this topic run inside the WSL Linux shell unless the step explicitly says PowerShell. This keeps the setup aligned with the Linux source deployment runbook and avoids native-Windows path ambiguity.

1. Install Linux prerequisites in the WSL distribution.

   ```
   sudo apt-get update
   sudo apt-get install -y git make curl python3 python3-venv python3-pip
   python3 --version
   ```

   Use the package manager and package names approved for your selected WSL distribution.

2. Clone BPM into the Linux home filesystem.

   ```
   cd ~
   git clone <repository-url> browser-policy-manager
   cd browser-policy-manager
   git checkout <approved-0.9.1-ref>
   ```

   Do not place the working checkout under `/mnt/c` for the supported WSL source workflow.

3. Create and activate the local Python environment.

   ```
   python -m venv .venv
   source .venv/bin/activate
   python -m pip install --upgrade pip
   pip install -e ".[dev]"
   ```

   If the distribution exposes Python only as `python3`, create the environment with `python3 -m venv .venv` and then use the activated environment.

4. Apply database migrations inside the WSL checkout.

   ```
   mkdir -p data
   export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
   alembic upgrade head
   ```

> Warning:
> The SQLite file is a Linux file inside WSL. Do not edit or move `data/bpm.db` from Windows tools while BPM is running.

The WSL checkout has Linux prerequisites, a repository-local virtual environment, installed dependencies, and a migrated local database.

Configure runtime variables and browser access before starting BPM.

Configure WSL runtime and browser access

Set BPM runtime variables inside WSL, choose a safe bind address, start the source process, and identify the browser URL from Windows.

The BPM checkout, virtual environment, dependencies, and migrations are ready inside the WSL Linux filesystem.

BPM reads `BPM_` environment variables from the Linux process. Modern WSL normally forwards `localhost` to Windows, but the WSL IP can be checked when browser access or firewall behavior is unclear.

1. Set local runtime variables in the WSL shell.

   ```
   source .venv/bin/activate
   export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
   export BPM_HOST="0.0.0.0"
   export BPM_PORT="8000"
   export BPM_RELOAD="true"
   ```

   `BPM_HOST="0.0.0.0"` allows the Windows browser to reach the WSL process through localhost forwarding. Use `127.0.0.1` only when the browser is also inside WSL or forwarding has been verified.

2. Start BPM from the WSL checkout.

   ```
   make dev
   ```

   The Makefile starts `uvicorn app.main:app --reload --port 8000`. Keep the WSL terminal open for startup messages, tracebacks, and file-watcher output.

3. Identify the Windows browser URL.

   ```
   hostname -I
   ```

   Try `http://localhost:8000/profiles` from Windows first. If localhost forwarding is unavailable, use the first WSL IP reported by `hostname -I`, for example `http://<wsl-ip>:8000/profiles`.

4. Keep security boundaries explicit.

   > Warning:
   > Opening the WSL port to other hosts, changing Windows firewall rules, adding TLS, or placing a reverse proxy in front of BPM is outside this WSL source-deployment topic and must wait for production-readiness guidance.

The BPM source process is running inside WSL and has an identified Windows browser access path.

Verify health, readiness, UI access, and WSL-specific failure modes.

Verify BPM source deployment through WSL

Check probes from WSL and Windows, confirm the UI route, inspect logs, and troubleshoot common WSL networking and filesystem issues.

BPM is running from the WSL checkout with the intended `BPM_DATABASE_URL`, host, port, and documentation-site settings.

Verification must prove both the Linux process response and the Windows browser access path. It does not certify native Windows installation, production hardening, reverse proxy behavior, backup readiness, or Firefox runtime enforcement.

1. Check health and readiness inside WSL.

   ```
   curl -fsS http://127.0.0.1:8000/health
   curl -fsS http://127.0.0.1:8000/health/ready
   ```

   Expected responses are `{"status":"ok"}` and `{"status":"ready","ready":true}`.

2. Check the same probes from Windows PowerShell.

   ```
   Invoke-WebRequest http://localhost:8000/health
   Invoke-WebRequest http://localhost:8000/health/ready
   ```

   If localhost does not work, retry with the WSL IP from `hostname -I`. Record which address succeeded.

3. Open the UI route from the Windows browser.

   Use `http://localhost:8000/profiles` or `http://<wsl-ip>:8000/profiles`. The Library route should load the BPM interface or an intentional empty/error state, not a traceback.

4. Troubleshoot WSL-specific connection and filesystem failures.

   For connection failures, confirm `make dev` is still running, check `BPM_HOST`, run `wsl --list --verbose`, and refresh the WSL IP after restarting the distribution. For slow file watching or permission errors, move the checkout from `/mnt/c` into `/home/<linux-user>`. If the issue remains, stop dependent integrations and provide the route, probe results, WSL state, and foreground logs to the BPM deployment owner.

   > Warning:
   > Passing health checks through WSL does not prove production readiness, Windows service behavior, managed backups, or HA support.

The administrator has evidence that BPM starts inside WSL, responds to probes, is reachable from Windows, and has no immediate WSL networking or filesystem blockers.

Keep PowerShell output, WSL shell commands, `BPM_*` variable names, probe responses, and foreground logs together for troubleshooting without copying secrets.

Operate and update BPM from source

Review DevOps configuration sources

Identify the current BPM configuration sources, record effective runtime values, and separate ordinary settings from unsupported secret-management expectations.

BPM is deployed from source on Linux or through WSL, and the administrator can inspect the shell environment, optional `.env` file, and source checkout.

BPM loads settings from environment variables with the `BPM_` prefix and from a repository-root `.env` file. BPM does not provide an application-managed secret store, credential vault, API token setting, encryption-key setting, or production configuration profile.

1. List the configuration sources used for this run.

   Record whether values come from shell exports, a local `.env`, or the documented defaults. Keep `.env` local to the checkout and do not commit it.

2. Record the runtime settings that affect process behavior.

   Include `BPM_APP_NAME`, `BPM_APP_VERSION`, `BPM_DEBUG`, `BPM_HOST`, `BPM_PORT`, `BPM_RELOAD`, `BPM_API_PREFIX`, `BPM_DEFAULT_LOCALE`, `BPM_SUPPORTED_LOCALES`, `BPM_I18N_DIR`, and `BPM_DOCUMENTATION_SITE_DIR`. Defaults are local-source defaults, not production policy.

3. Record data, schema, and network-related settings separately.

   Track `BPM_DATABASE_URL`, `BPM_DB_ECHO`, `BPM_ENABLE_CORS`, `BPM_CORS_ALLOW_ORIGINS`, `BPM_SCHEMA_BASE_URL`, `BPM_SCHEMA_CACHE_DIR`, and `BPM_SCHEMA_HTTP_TIMEOUT`. Treat database URLs and network origins as operational evidence even when they contain no password.

4. Keep secrets out of BPM configuration examples.

   > Warning:
   > BPM source deployment documentation does not provide authentication setup, managed secrets, encryption-at-rest configuration, token rotation, tenant isolation, or secret redaction guarantees. If your environment needs those controls, record them as external platform controls rather than BPM-supported features.

The deployment has a reviewed configuration inventory that matches current source settings and does not imply unsupported secret-management behavior.

Use the storage, network, and operational-boundary topics to review the impact of the recorded values before exposing BPM beyond a controlled local environment.

Plan storage, logs, and backup evidence

Locate the source-deployment database, schema cache, documentation site, foreground logs, and manual backup evidence without claiming managed backup or restore support.

The deployment configuration inventory includes `BPM_DATABASE_URL`, `BPM_SCHEMA_CACHE_DIR`, and `BPM_DOCUMENTATION_SITE_DIR`.

The default source deployment stores application data in SQLite at `sqlite+aiosqlite:///./data/bpm.db`, corresponding to `data/bpm.db` under the repository root. Schema cache files live under `app/schemas/mozilla` by default, and an installed documentation artifact is read from `app/documentation/site` when present.

1. Identify the files and directories that hold operational state.

   Record the repository path, `data/`, `data/bpm.db`, `app/schemas/mozilla`, and `app/documentation/site`. In WSL, keep these paths in the Linux filesystem rather than under `/mnt/c`.

2. Define a manual backup evidence step before risky work.

   Stop the foreground BPM process, copy `data/bpm.db` to a controlled backup location, and record timestamp, source revision, database URL, file size, and checksum. BPM does not provide automated backup, point-in-time recovery, or tested restore orchestration.

3. Capture logs from the current source process.

   Source runs write startup messages, Uvicorn access output, tracebacks, migration errors, port conflicts, and documentation-site warnings to the foreground terminal, stdout, or stderr. No dedicated BPM application log directory, log rotation policy, request ID, audit log, or Event Viewer integration is documented in this release.

4. Pair file backup evidence with export evidence when profile handoff matters.

   Use existing API export examples to export relevant Firefox `policies.json` documents and store response status, profile ID, revision, schema channel, and exported-document hash. Do not treat export evidence as a complete database restore.

The administrator knows where current source-deployment state lives and which manual evidence can be collected before updates or troubleshooting.

Before changing source revisions or database schemas, use the current update procedure and stop when backup or restore evidence is incomplete.

Review network, CORS, and security limits

Review host, port, reload, health probes, CORS defaults, and security limits before making a source-run BPM instance reachable.

The source process can be started with `make dev`, and the administrator can change `BPM_HOST`, `BPM_PORT`, `BPM_RELOAD`, `BPM_ENABLE_CORS`, and `BPM_CORS_ALLOW_ORIGINS`.

Current defaults are optimized for local source operation: `BPM_HOST="0.0.0.0"`, `BPM_PORT="8000"`, `BPM_RELOAD="true"`, `BPM_ENABLE_CORS="true"`, and permissive origins equivalent to `["*"]`. These defaults are not a hardened network exposure policy.

1. Choose the narrowest bind address for the current environment.

   Use `127.0.0.1` for local Linux checks. Use `0.0.0.0` only when WSL browser forwarding or a controlled lab network requires it. Record the reason for any non-loopback binding.

2. Gate operational access with health and readiness probes.

   Use `GET /health` and `GET /health/ready` before API examples or UI smoke checks. Successful probes show process liveness/readiness only; they do not prove database backup status, Firefox enforcement, authentication, authorization, or production topology.

3. Restrict CORS for any shared evaluation.

   For local experiments, permissive `BPM_CORS_ALLOW_ORIGINS` may be acceptable. For a shared host, set explicit origins and record who owns the browser origin. BPM documentation does not define credentialed CORS, CSRF protections, API authentication, or tenant isolation.

4. Keep unsupported security controls out of the BPM claim.

   > Warning:
   > This release does not ship TLS termination, reverse proxy recipes, firewall policy, rate limiting, request signing, audit logging, SSO, OAuth/OIDC setup, role-based access control, vulnerability scanning, or HA failover. Document those as external controls if your environment adds them.

The source-run BPM network posture is documented as a controlled local or lab exposure rather than a production security architecture.

If probes fail or exposure is unclear, use the source-deployment verification topics before continuing with API integration examples.

Record source-operation boundaries

Create an operations handoff note that states the runtime user, process model, checks, and external responsibilities.

The configuration, storage, backup, network, and CORS reviews have been completed for the source-run instance.

Use this guide for a source-run BPM instance. Record the platform components and operating procedures owned by your organization alongside the BPM workflow.

1. Record the runtime user and process model.

   Identify the shell user that owns the checkout, `.venv`, `data/`, and the running `make dev` or `uvicorn app.main:app --reload --port 8000` process. Do not claim a supported service account, systemd unit, Windows service, or restart policy unless your external platform provides and owns it.

2. Record the checks that are supported today.

   Include source revision, active configuration values, migration status, `/health`, `/health/ready`, UI route smoke, API example status, foreground-log review, and manual backup evidence. Link API evidence to reusable API examples rather than duplicating endpoint procedures.

3. Record external responsibilities.

   Identify the organization-owned procedures for services, reverse proxy and TLS, secrets, authentication, backups, restore, monitoring, log retention, and production hardening.

4. Attach user-facing recovery references without moving ownership.

   When an operator needs to understand UI symptoms, link to user-facing troubleshooting such as product connection recovery. Keep deployment, API, backup, and network ownership in the Administrator/DevOps Guide.

The handoff note records the source deployment, its evidence, and the responsibilities owned by the operating organization.

Use this note with the update-from-source runbook and the organization's operating procedures.

Prepare evidence before updating from source

Collect the source revision, configuration, database backup, profile export evidence, and stop conditions before moving BPM to a newer source revision.

BPM is running from a Linux or WSL source checkout, and the administrator has completed the configuration, storage, backup, and operational-boundary reviews.

An update-from-source procedure changes code, dependencies, database schema, and documentation output expectations. BPM does not provide automatic rollback, database downgrade guarantees, managed backup, or packaged upgrade tooling. Treat evidence collection as a required pre-update gate.

1. Stop the foreground BPM process and record the current source state.

   ```
   git status --short
   git rev-parse --short HEAD
   ```

   Continue only when the working tree state is understood and the current source revision is recorded as the rollback reference.

2. Record effective configuration and runtime paths.

   Capture the active `BPM_DATABASE_URL`, `BPM_DOCUMENTATION_SITE_DIR`, `BPM_SCHEMA_CACHE_DIR`, `BPM_HOST`, `BPM_PORT`, and whether values came from shell exports or `.env`. Do not copy secrets into the update note.

3. Create manual database backup evidence while BPM is stopped.

   ```
   mkdir -p backups
   cp data/bpm.db backups/bpm-pre-update.db
   sha256sum backups/bpm-pre-update.db
   ```

   Record timestamp, current revision, database URL, file size, and checksum. If the deployment does not use `data/bpm.db`, back up the path resolved from `BPM_DATABASE_URL` instead.

4. Export critical profile evidence before changing the source revision.

   Use the existing API export examples to save required Firefox `policies.json` documents and record response status, profile ID, revision, schema channel, and exported-document hash.

   > Warning:
   > If database backup or export evidence is missing, stop the update. Do not replace missing backup evidence with an assumption that Git rollback or `alembic downgrade` will be safe.

The update has a verified pre-change evidence set and a known previous source reference.

Proceed to refreshing the source revision and dependencies only after the evidence set is complete and reviewed.

Refresh the source revision and dependencies

Move the checkout to an approved newer revision and reinstall the Python environment without hiding dependency or worktree failures.

The pre-update evidence set exists, the previous revision is recorded, and an approved target branch, tag, or commit is known.

BPM source updates are Git and Python-environment operations performed in the existing checkout. The procedure does not install a packaged distribution, native service, or production upgrade manager.

1. Fetch the approved source reference.

   ```
   git fetch --tags
   git status --short
   ```

   Stop if local changes are unexpected. Preserve user-owned or site-owned changes instead of overwriting them.

2. Move to the approved newer source revision.

   ```
   git checkout <approved-new-ref>
   git rev-parse --short HEAD
   ```

   Record the target revision and compare it with the approved change ticket or release note.

3. Refresh the repository-local Python environment.

   ```
   source .venv/bin/activate
   python -m pip install --upgrade pip
   pip install -e ".[dev]"
   ```

   Use the existing `.venv` unless the update note explicitly requires rebuilding it. Record dependency failures before changing the database.

4. Stop on unresolved source or dependency failures.

   > Warning:
   > If checkout, dependency installation, or virtual-environment activation fails, do not run migrations. Return to the recorded previous revision only if the database has not been changed, then reinstall the previous dependency set and re-run the pre-update smoke checks.

The checkout and Python environment are on the approved target revision, or the update has stopped before database changes.

Run database migrations and documentation checks only after the dependency refresh succeeds.

Run migrations and source-update checks

Apply database migrations and verify the updated BPM source deployment after the source revision changes.

The checkout and dependencies have been refreshed successfully, and the database backup evidence was collected while BPM was stopped.

Updates may introduce database migrations. BPM documents forward migration with `alembic upgrade head`; it does not promise automatic downgrade or safe schema rollback.

1. Apply database migrations to the intended database.

   ```
   export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
   alembic upgrade head
   ```

   Use the actual deployment value for `BPM_DATABASE_URL`. Stop immediately if migration output is unclear or fails.

2. Start the updated BPM process and verify its current interface.

   Check health and readiness, then open the Library route. If documentation is required for the deployment, confirm that `/help/` reports a ready installed artifact rather than a missing or incompatible state.

3. Stop on migration or runtime verification failures.

   > Warning:
   > After a migration has run, do not assume Git checkout rollback is enough. Do not run `alembic downgrade` unless a reviewed release note explicitly says it is supported for the exact revision pair. Use the pre-update database backup as the safe recovery baseline when rollback is required.

The updated source revision has completed migrations and runtime verification, or the update has stopped at a named failure point.

Start BPM and run the post-update verification topic before handing the instance back to users or integrators.

Verify the update and apply rollback stop conditions

Start the updated BPM process, verify probes, UI, API evidence, and documentation status, then decide whether to accept, stop, or restore from backup.

Migrations, source smoke, and documentation rebuild checks have completed or a failure point has been recorded.

Post-update verification proves that the updated source run is usable for the current controlled environment. It does not prove production readiness, reverse proxy behavior, HA, managed backup, or Firefox runtime enforcement.

1. Start the updated source process and check probes.

   ```
   make dev
   curl -fsS http://127.0.0.1:8000/health
   curl -fsS http://127.0.0.1:8000/health/ready
   ```

   Expected responses remain `{"status":"ok"}` and `{"status":"ready","ready":true}`.

2. Verify the product UI, API evidence, and documentation status.

   Open `http://127.0.0.1:8000/profiles`, run the required reusable API examples, confirm exported `policies.json` evidence where needed, and check that `/help/` does not report stale, incompatible, or missing documentation for the installed artifact.

3. Accept the update only when every required check is recorded.

   Record previous revision, target revision, migration result, source smoke result, documentation validation/build result, health/readiness responses, UI smoke result, API/export evidence, and any known deferred production controls.

4. Apply rollback or stop conditions without inventing automatic recovery.

   If failure occurs before migrations, return to the previous source revision and reinstall dependencies. If failure occurs after migrations, stop BPM, restore the verified pre-update database backup, return to the previous source revision, reinstall dependencies, and repeat health/UI smoke checks. If backup evidence is missing, restore is unverified, or data changed after migration, stop and escalate instead of claiming rollback success.

   > Warning:
   > Do not mark the update complete when probes fail, documentation is stale, exports differ unexpectedly, migrations are uncertain, or rollback evidence is incomplete.

The update is either accepted with complete evidence, stopped at a known blocker, or restored from the pre-update backup with explicit proof.

Use the resulting evidence in the operations handoff note when recording platform-owned production, HA, and reverse-proxy controls.

Check BPM health and readiness

Use the current liveness and readiness probes as a lightweight operational handshake before API workflows.

Set `$BPM_BASE_URL`. For host setup, service start, logs, updates from source, and operating-system details, use the source-deployment Administrator Guide; this API topic only documents the integration handshake.

This task covers `API-HEALTH-001` and `API-HEALTH-002`. BPM exposes prefix-free probes at `/health` and `/health/ready`. BPM does not expose dependency diagnostics through these probes; they are not versioned and do not replace product-specific retry or incident policy.

1. Call the liveness probe before starting a short API sequence.

   Request example: `GET $BPM_BASE_URL/health`. Response example: `200 {"status":"ok"}`. Error example: no application error is currently declared; transport, gateway, or process failures are handled outside BPM.

2. Call the readiness probe before operations that depend on application startup being complete.

   Request example: `GET $BPM_BASE_URL/health/ready`. Response example: `200 {"status":"ready","ready":true}`. Error example: no degraded-state or dependency-detail body is currently exposed.

3. Use readiness as a gate, not as a complete dependency inventory.

   When `ready` is `true`, the integrating product may continue with discovery, validation, import, export, or profile lifecycle calls. It must not infer database health details, schema cache details, documentation portal status, queue state, or deployment topology from this response.

4. Keep polling and retry decisions in the integrating product.

   BPM does not declare a polling interval, retry-after header, service-level objective, circuit-breaker contract, request ID, or webhook for probe changes. Store your own timestamp, observed status, timeout, and retry decision.

5. Separate API handshake checks from source deployment troubleshooting.

   If probes are unreachable or the response differs from the examples, switch to the source-deployment Administrator Guide for process, environment, database, log, and update-from-source checks instead of expanding this API integration task into deployment instructions.

6. Record the operational handshake result with the downstream API action.

> Warning:
> A successful health or readiness response does not authenticate callers, authorize operations, reserve profile revisions, guarantee later API success, or prove Firefox runtime enforcement. Treat it as one small preflight signal.

The integrating product has a minimal and current BPM operational handshake for liveness and readiness without depending on undocumented deployment behavior.

After a successful readiness check, continue with OpenAPI discovery or the specific API workflow topic for validation, import/export, or profile lifecycle operations.

Integrate with the BPM API

Integration audience and API boundary

The API integration section of the Administrator/DevOps Guide is for external tools that use BPM's current programmatic API without assuming unsupported product guarantees.

Audience

Use this guide when you build or review a configuration, compliance, orchestration, inventory, or control product that exchanges profile and Firefox policy data with BPM. The primary readers are API integrators, security reviewers, and release engineers who need a repeatable contract for profile synchronization, validation gates, policy import/export, compliance metadata, inventory, and health checks.

Evidence boundary

This section documents the API BPM currently exposes. It does not authorize new endpoints, security models, data schemas, retry semantics, or compatibility promises.

Supported consumer roles

Supported consumers are generic roles: configuration manager, compliance scanner, policy review gate, profile inventory collector, deployment orchestrator, and operational monitor. They can call service, profile, Firefox import/export, validation, and health operations listed in the API inventory.

Unsupported claims

Do not describe a product-specific connector, marketplace integration, certified integration, partnership, managed agent, or vendor workflow unless that connector actually ships. HTML routes, static assets, generated schema UIs, and underscore-prefixed Python helpers are not stable machine-integration APIs.

Data ownership

BPM owns the normalized profile model, validation state, revision, lifecycle state, opaque compliance object, and canonical Firefox export rendering. Integrating products own their scheduling, authorization, audit trail, retry policy, exception record, deployment decision, and any runtime verification outside BPM.

Supported integration patterns

BPM supports generic API patterns for profile, validation, Firefox document, metadata, inventory, and health workflows.

Profile synchronization

An integrating product can list profiles, read a profile by ID, create a normalized profile, patch mutable profile fields, archive, restore, permanently delete, or reset the Library. Synchronization must treat profile ID and revision as BPM-owned state and must not assume bulk operations, name mutation, or automatic rollback across multiple requests.

Validation gates

A validation gate can submit a candidate canonical `policies.json` document to the supported validation endpoint before a deployment workflow accepts it. The gate must inspect the response body as well as the status code, because validation can return a successful HTTP status with `ok=false`.

Policy import and export

An integration can import Firefox `policies.json` as a BPM profile and export a saved BPM profile back to canonical Firefox JSON. Do not interchange the normalized profile model with canonical Firefox document shape; import/export is the boundary between those representations.

Compliance metadata handoff

The API exposes an opaque `compliance` object on profile create, update, and import operations. Integrations may pass their own metadata through BPM, but BPM does not publish a versioned schema for that object and does not validate external compliance semantics.

Inventory and health checks

Inventory-style consumers can list profiles, request stats for the same filter context, read individual profiles, and call service, liveness, and readiness endpoints. Health responses are intentionally compact and do not expose per-dependency diagnostics, documentation status, or deployment readiness beyond current fixed fields.

Unsupported patterns

The guide does not document authentication, authorization, rate limits, idempotency keys, request IDs, webhooks, bulk export, bulk CRUD, cursor pagination, product-specific connectors, vendor partnerships, or transactional orchestration because BPM does not expose those guarantees.

API conventions

Use these conventions when reading BPM OpenAPI contracts and building integration requests.

Base URLs and discovery

Examples use `$BPM_BASE_URL` as a deployment value. Discover the current contract from `/openapi.json` and, for interactive inspection only, the FastAPI schema UIs at `/docs` and `/redoc`. Product documentation is available at `/help/`; do not use `/docs` as its route.

Content types

Most programmatic operations use JSON request and response bodies. Firefox import accepts `application/json` and `multipart/form-data`; unsupported media types can return `415`. Firefox export returns `application/json` and can add `Content-Disposition` when download output is requested.

Identifiers and lifecycle

Profile IDs are integer path identifiers owned by BPM. Store the returned `id` and `revision` after create, import, read, and update operations. Profile names are unique and cannot be changed through `ProfileUpdate`. Lifecycle values are active, archived, and all; `include_deleted` is a compatibility flag that must be interpreted together with lifecycle.

Models and schema channels

`ProfileCreate`, `ProfileUpdate`, and `ProfileRead` are BPM normalized profile shapes. Firefox import/export uses canonical `policies.json` with top-level `policies`. Validation uses `ValidationRequest` and supported schema channels such as Firefox ESR and Release; a plain policy mapping remains compatibility behavior, not the preferred external shape.

Filtering, sorting, and pagination

Profile list and stats operations support search, schema, validation state, lifecycle, and include-deleted filters. List pagination is offset/limit with default `limit=50`, maximum `200`, and `offset=0`. Sorting uses current described fields such as `created_at`, `updated_at`, `name`, `schema_version`, and `id`, with `asc` or `desc` order.

Status and error handling

Use status codes and body shape together. `200`, `201`, and `204` are success families, but validation can return `200` with `ok=false`. Error bodies use FastAPI `detail` and may contain a string, a validation list, or a BPM object with `message`, `error`, or `issues`.

Current API limitations and security constraints

BPM documents current limitations plainly so integrations do not infer guarantees that are not exposed.

Security constraints

The current API has no authentication or authorization layer. CORS defaults to `*` origins unless deployment settings override it. Documentation must not imply API token authentication, OAuth, role-based access control, tenant isolation, or production authorization behavior.

Versioning and compatibility

The API has no version prefix, compatibility policy, deprecation header, or media-type version. Integrations should pin the BPM release they test against and watch `/openapi.json`, the API inventory, and release notes rather than assuming guaranteed backward compatibility.

Concurrency, idempotency, and retry

Only PATCH supports optimistic concurrency through optional `expected_revision`. Create, import, delete, restore, reset, and export do not expose ETags, precondition headers, idempotency keys, or retry contracts. Consumers must design their own safe retry and duplicate-name recovery behavior.

Bulk and transaction boundaries

BPM exposes no bulk profile CRUD endpoint, no bulk export endpoint, no cursor pagination, and no multi-operation transaction contract. A failed multi-step integration is not automatically rolled back by BPM; the integrating product owns recovery sequencing and audit records.

Error envelope limitations

Error responses do not use one stable versioned envelope. Status `400`, `404`, `409`, `415`, `422`, and `503` can carry different `detail` shapes. Consumers must branch by operation, status, and documented body shape.

Excluded and HTML surfaces

The six OpenAPI `web` operations are HTML product routes, not stable machine-integration APIs. `/i18n/{locale}.json`, `/favicon.ico`, `/static/*`, generated schema UIs, and underscore-prefixed Python helpers are excluded from integration operation topics.

Health response limits

`/health` and `/health/ready` expose compact status fields only. Readiness does not report per-dependency diagnostics, database details, schema health, documentation portal status, or deployment approval. Operational monitors should treat them as minimal handshake endpoints.

Reuse DevOps API examples and environment templates

Use language-neutral request patterns, environment templates, copyable curl commands, multipart samples, and compact Python assertions for current BPM API workflows.

Set `BPM_BASE_URL` for the BPM instance under test. Do not put credentials, tenant names, personal host names, private hosts, or production profile IDs into reusable examples; BPM does not currently document API authentication.

This task covers `API-HEALTH-001`, `API-HEALTH-002`, `API-PROFILE-001`, `API-PROFILE-003`, `API-PROFILE-004`, `API-PROFILE-005`, `API-FF-001`, `API-VAL-001`, and `API-FF-002`. The examples intentionally use `$BPM_BASE_URL`, synthetic names, `release-153`, response assertions, and disposable fixture IDs instead of environment-specific hosts or secrets.

1. Create a reusable environment template before copying curl requests.

   Request example: `export BPM_BASE_URL="${BPM_BASE_URL:?set BPM_BASE_URL first}"`. Response example: the shell keeps one base URL, schema channel, synthetic profile name, and optional import path for the whole run. Error example: the shell exits the assignment when a required value is missing.

   ```
   # .env.devops-example.template
   BPM_BASE_URL=
   BPM_SCHEMA_CHANNEL=release-153
   BPM_PROFILE_NAME=docs-api-example
   BPM_JSON_IMPORT_PATH=./policies.json

   export BPM_BASE_URL="${BPM_BASE_URL:?set BPM_BASE_URL first}"
   export BPM_SCHEMA_CHANNEL="${BPM_SCHEMA_CHANNEL:-release-153}"
   export BPM_PROFILE_NAME="${BPM_PROFILE_NAME:-docs-api-example}"
   export BPM_JSON_IMPORT_PATH="${BPM_JSON_IMPORT_PATH:-./policies.json}"
   ```

2. Gate every reusable sequence with health and readiness probes.

   Request example: `curl -fsS "$BPM_BASE_URL/health"` and `curl -fsS "$BPM_BASE_URL/health/ready"`. Response example: `{"status":"ok"}` and `{"status":"ready","ready":true}`. Error example: curl exits non-zero for transport errors or non-2xx responses when `-f` is used.

   ```
   curl -fsS "$BPM_BASE_URL/health"
   curl -fsS "$BPM_BASE_URL/health/ready"
   ```

3. Reuse profile lifecycle snippets for create, list, read, and guarded update.

   Request example: `POST $BPM_BASE_URL/api/profiles`, `GET $BPM_BASE_URL/api/profiles`, `GET $BPM_BASE_URL/api/profiles/$PROFILE_ID`, and `PATCH $BPM_BASE_URL/api/profiles/$PROFILE_ID`. Response example: `201 ProfileRead`, `200`

`ProfileRead[]`, `200 ProfileRead`, and an incremented `revision`. Error examples: `400`, `404`, `409` stale `expected_revision`, or `422`.

```
PROFILE_JSON="$(curl -fsS -H 'Content-Type: application/json' \
  -d '{"name":"docs-api-example","schema_version":"release-153","flags":{"DisableTelemetry":true}}' \
  "$BPM_BASE_URL/api/profiles")"
PROFILE_ID="$(python -c 'import json,sys; print(json.load(sys.stdin)["id"])' <<<"$PROFILE_JSON")"
PROFILE_REVISION="$(python -c 'import json,sys; print(json.load(sys.stdin)["revision"])'
<<<"$PROFILE_JSON")"

curl -fsS "$BPM_BASE_URL/api/profiles?q=docs-api-example&lifecycle=active&limit=50&offset=0"
curl -fsS "$BPM_BASE_URL/api/profiles/$PROFILE_ID"
curl -fsS -X PATCH -H 'Content-Type: application/json' \
  -d "{\"flags\":
{\"DisableTelemetry\":true,\"BlockAboutConfig\":true},\"expected_revision\":$PROFILE_REVISION}" \
  "$BPM_BASE_URL/api/profiles/$PROFILE_ID"
```

4. Reuse validation and JSON import snippets with explicit payload shape.

   Request example: `POST $BPM_BASE_URL/api/validate/release-153` and `POST $BPM_BASE_URL/api/profiles/import/firefox/policies.json`. Response example: `200 {"ok":true,"profile":"release-153"}` and `201 ProfileRead`. Error examples: `400`, `409`, `415`, `422`, or `503`.

```
curl -fsS -H 'Content-Type: application/json' \
  -d '{"document":{"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}}' \
  "$BPM_BASE_URL/api/validate/$BPM_SCHEMA_CHANNEL"

curl -fsS -H 'Content-Type: application/json' \
  -d '{"name":"docs-json-import-example","schema_version":"release-153","compliance":{"source":"docs-
devops-template"},"document":{"policies":{"DisableTelemetry":true}}}' \
  "$BPM_BASE_URL/api/profiles/import/firefox/policies.json"
```

5. Reuse multipart import and export snippets for file-based handoff.

   Request example: `multipart/form-data` upload to `POST $BPM_BASE_URL/api/profiles/import/firefox/policies.json` followed by `GET $BPM_BASE_URL/api/export/profiles/$PROFILE_ID/firefox/policies.json?pretty=1`. Response example: `201 ProfileRead` for import and `200 application/json` containing canonical Firefox `policies` for export. Error example: `404` missing profile or `422` invalid profile ID.

```
curl -fsS \
  -F "name=docs-multipart-import-example" \
  -F "schema_version=$BPM_SCHEMA_CHANNEL" \
  -F 'compliance={"source":"docs-devops-template"}' \
  -F "file=@$BPM_JSON_IMPORT_PATH;type=application/json" \
  "$BPM_BASE_URL/api/profiles/import/firefox/policies.json"

curl -fsS "$BPM_BASE_URL/api/export/profiles/$PROFILE_ID/firefox/policies.json?pretty=1"
```

6. Use a compact Python helper when the integration needs reusable response assertions.

Request example: `session.post(f"{base_url}/api/validate/{schema_channel}", json=payload, timeout=10)`. Response example: the helper raises immediately unless the status and JSON body match the assertion. Error example: assertion failures, timeout exceptions, stale `expected_revision`, or structured API errors must be recorded by the integrating product.

```
import os
import requests

base_url = os.environ["BPM_BASE_URL"].rstrip("/")
schema_channel = os.environ.get("BPM_SCHEMA_CHANNEL", "release-153")
session = requests.Session()

ready = session.get(f"{base_url}/health/ready", timeout=10)
ready.raise_for_status()
assert ready.json() == {"status": "ready", "ready": True}

payload = {"document": {"policies": {"DisableTelemetry": True, "BlockAboutConfig": True}}}
validation = session.post(f"{base_url}/api/validate/{schema_channel}", json=payload, timeout=10)
validation.raise_for_status()
assert validation.json() == {"ok": True, "profile": schema_channel}

created = session.post(
    f"{base_url}/api/profiles",
    json={"name": "docs-python-example", "schema_version": schema_channel, "flags":
payload["document"]["policies"]},
    timeout=10,
)
created.raise_for_status()
profile = created.json()
assert profile["schema_version"] == schema_channel

exported = session.get(
    f"{base_url}/api/export/profiles/{profile['id']}/firefox/policies.json?pretty=1",
    timeout=10,
)
exported.raise_for_status()
assert exported.json() == {"policies": payload["document"]["policies"]}
```

> Warning:
> Keep response status, profile ID, revision, schema channel, exported document hash, and assertion result in external evidence. Do not copy real host names, secrets, API keys, secret values, or production identifiers into documentation examples.

The integrating product has reusable curl and Python examples that are safe to copy, independent of deployment hosts, synchronized with current BPM API behavior, and tied to explicit response assertions.

Combine these examples with the pull/list/read, validate-before-apply, import-review-export, and failure-audit runbooks when building full external control-product workflows.

Synchronize profile lifecycle data

Use profile list, stats, read, create, and update endpoints without assuming bulk or transactional behavior.

Set `$BPM_BASE_URL` and read `/openapi.json` for the current ProfileCreate, ProfileUpdate, and ProfileRead shapes.

This task covers `API-PROFILE-001`, `API-PROFILE-002`, `API-PROFILE-003`, `API-PROFILE-004`, and `API-PROFILE-005`. Store returned `id` and `revision`; do not infer identity from name alone.

1. List profiles with filters before choosing a profile to synchronize.

   Request example: `GET $BPM_BASE_URL/api/profiles?q=corp&lifecycle=active&limit=50&offset=0&sort=updated_at&order=desc`. Response example: `200 ProfileRead[]`. Error example: `422` for invalid typed or ranged query input.

2. Read profile statistics for the same filter context when you need counts.

   Request example: `GET $BPM_BASE_URL/api/profiles/stats?q=corp&lifecycle=active`. Response example: `200 {"filtered":1,"total":3}`. Error example: `422` for invalid query input.

3. Read the selected profile by integer ID and include archived records only when required.

   Request example: `GET $BPM_BASE_URL/api/profiles/42?include_deleted=false`. Response example: `200 ProfileRead` with `id`, `revision`, timestamps, lifecycle, and validation state. Error example: `404` for a missing or not-visible profile; `422` for invalid path or query input.

4. Create a normalized BPM profile only when the integration owns the new profile name.

   Request example: `POST $BPM_BASE_URL/api/profiles` with JSON `{"name":"corp-baseline","schema_version":"release-153","flags":{},"compliance":{}}`. Response example: `201 ProfileRead`. Error examples: `400` invalid schema/profile, `409` duplicate name, or `422` request or policy validation.

5. Update mutable fields with PATCH and send `expected_revision` when you need optimistic concurrency.

   Request example: `PATCH $BPM_BASE_URL/api/profiles/42` with JSON `{"description":"Reviewed by control plane","expected_revision":7}`. Response example: `200 ProfileRead` with incremented revision after a material update. Error examples: `400` invalid schema/profile, `404` missing or archived profile, `409` stale `expected_revision`, or `422` validation error.

6. Record the synchronized profile ID, revision, validation state, schema channel, and any opaque compliance metadata in the integrating product.

> Warning:
> BPM does not provide a multi-operation transaction. If a later step fails, the integrating product owns recovery, audit records, and retry decisions.

The integration has a reproducible profile synchronization path for list, stats, read, create, and update operations.

Use the destructive lifecycle task before archive, restore, permanent delete, or reset operations.

Archive, restore, permanently delete, or reset profiles

Use destructive profile lifecycle endpoints with explicit review and recovery records.

Confirm the target profile ID, current lifecycle state, and external approval record before calling destructive endpoints.

This task covers `API-PROFILE-006`, `API-PROFILE-007`, `API-PROFILE-008`, and `API-PROFILE-009`. BPM has no bulk confirmation token, no undo endpoint for permanent delete, and no transaction rollback across operations.

1. Archive a profile when you need to remove it from active workflows while preserving it for review.

   > Warning:
   > Archive is a destructive lifecycle change for active workflows. Record the approval reason and profile ID before the call.

   Request example: `DELETE $BPM_BASE_URL/api/profiles/42`. Response example: `204` with no JSON body. Error examples: `404` missing profile or `422` invalid path input.

2. Restore an archived profile only after checking whether the name and profile purpose are still valid.

   Request example: `POST $BPM_BASE_URL/api/profiles/42/restore`. Response example: `200 ProfileRead`. Error examples: `404` profile cannot be restored or found; `422` invalid path input.

3. Permanently delete a profile only when the external retention policy allows removal.

   > Warning:
   > Permanent delete removes an active or archived profile and has no BPM undo endpoint. Export or record required evidence before the call.

   Request example: `DELETE $BPM_BASE_URL/api/profiles/42/hard`. Response example: `204` with no JSON body. Error examples: `404` missing profile or `422` invalid path input.

4. Reset the Library only in disposable or explicitly approved environments.

   > Warning:
   > Reset permanently deletes every profile in the Library and currently requires no confirmation token. Never use it as a routine synchronization shortcut.

   Request example: `DELETE $BPM_BASE_URL/api/profiles/reset`. Response example: `200 {"deleted":3}`. Error behavior: no application error is currently declared; database failures use framework handling.

5. After any destructive call, re-list profiles and read the affected record when applicable.

   Request example: `GET $BPM_BASE_URL/api/profiles?lifecycle=all`. Response example: `200 ProfileRead[]`. Error example: `422` invalid query input. Use this verification to update the integrating product's inventory state.

6. Write the external audit record with operation ID, HTTP method, path, profile ID, approver, reason, timestamp, response status, and recovery decision.

   Expected result: a later reviewer can distinguish archive, restore, hard delete, and reset actions and can see why the integration considered the operation safe.

Destructive profile lifecycle operations are documented with request, response, error, warning, and recovery evidence.

If the approval record is missing, stop the integration workflow before calling permanent delete or reset.

Import Firefox policies.json through the API

Create a normalized BPM profile from a canonical Firefox Enterprise `policies.json` document using JSON or multipart input.

Set `$BPM_BASE_URL`, choose a supported schema channel such as `release-153`, and prepare a full Firefox document with a top-level `policies` object.

This task covers `API-FF-001`. Import accepts `application/json` with `FirefoxPoliciesJsonImportRequest` or `multipart/form-data` with a `file` field. BPM validates the canonical Firefox document, normalizes `policies` into profile `flags`, and stores optional opaque `compliance` metadata on the BPM profile.

1. Submit a JSON import request when the integrating product already has the document in memory.

   Request example: `POST $BPM_BASE_URL/api/profiles/import/firefox/policies.json` with `Content-Type: application/json` and JSON `{"name":"api-json-import","description":"Imported by control product","schema_version":"release-153","compliance":{"framework":"cis","layer":"cis_l1"},"document":{"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}}`.

2. Use multipart import when the source system hands over a file.

   Request example: `POST $BPM_BASE_URL/api/profiles/import/firefox/policies.json` with `multipart/form-data`, fields `name=api-multipart-import`, `schema_version=release-153`, optional `description`, optional `compliance={"framework":"cis","layer":"cis_l2"}` as a JSON string, and `file=@policies.json;type=application/json`. Runtime also accepts a multipart `document` field, but the OpenAPI multipart schema currently declares `file`; use `file` for new examples.

3. Read the created BPM profile from the `201 ProfileRead` response.

   Response example: `201 ProfileRead` with `schema_version="release-153"`, `flags.DisableTelemetry=true`, `flags.BlockAboutConfig=true`, stored `compliance`, `id`, and `revision`. Store the returned profile ID instead of inferring identity from the profile name.

4. Treat import errors as profile creation failures and do not assume partial profile state.

   Error examples: `400` for malformed JSON, wrong document shape, or unsupported schema channel; `409` for duplicate profile name; `415` for unsupported media type; `422` for request or Firefox policy validation errors.

5. Keep canonical Firefox and normalized BPM shapes separate in integration code.

   The request `document.policies` is the Firefox deployment boundary. The created profile's `flags` are BPM's normalized model. Do not send a top-level `flags` object to this import endpoint.

6. Record import evidence in the integrating product.

> Warning:
> Import creates a BPM profile but does not deploy Firefox settings, prove runtime enforcement, or provide an idempotency key. Record the source document hash, schema channel, returned profile ID, revision, compliance metadata, and duplicate-name recovery decision externally.

The Firefox `policies.json` document is represented as a normalized BPM profile that can be reviewed, validated, exported, and tracked by ID.

Use the export task to render the saved profile back to canonical Firefox JSON before handing it to deployment tooling.

Export canonical Firefox policies.json through the API

Render one BPM profile as the Firefox Enterprise deployment document with optional download and pretty-print parameters.

Store the BPM profile ID returned by create or import. If the profile is archived, decide explicitly whether the integration is allowed to export it with `include_deleted=true`.

This task covers `API-FF-002`. Export reads BPM's normalized `flags` and returns canonical Firefox JSON with a top-level `policies` object. BPM profile metadata and opaque `compliance` metadata are not exported into the Firefox deployment document.

1. Export the active profile as canonical Firefox JSON.

   Request example: `GET $BPM_BASE_URL/api/export/profiles/42/firefox/policies.json`. Response example: `200 application/json` with JSON `{"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}`.

2. Add download mode when the caller needs an attachment filename.

   Request example: `GET $BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?download=1`. Response example: `200 application/json` plus `Content-Disposition: attachment; filename="profile-42-policies.json"`. The JSON body is the same deployment document.

3. Choose formatting parameters deliberately.

   Request example: `GET $BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?pretty=1` selects indentation of two spaces when `indent` is omitted. Request example: `GET $BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?indent=0` uses the explicit indentation parameter. Error example: `422` for invalid constrained query values.

4. Export an archived profile only when the external workflow has a review reason.

   Request example: `GET $BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?include_deleted=true&download=1&pretty=1`. Response example: `200 application/json`. Error examples: `404` when a missing or archived profile is not visible; `422` for invalid path or query input.

5. Validate the exported document before handing it to another system.

   Request example: `POST $BPM_BASE_URL/api/validate/release-153` with JSON `{"document":{"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}}`. Response example: `200 {"ok":true,"profile":"release-153"}`. Validation confirms schema shape, not Firefox runtime deployment success.

6. Record the exact exported artifact externally.

> Warning:
> Export has no ETag, precondition header, bulk endpoint, or transaction wrapper. Record the profile ID, revision observed before export, schema channel, query parameters, exported document hash, and downstream handoff decision outside BPM.

The integration has a canonical Firefox `policies.json` artifact that excludes BPM-only metadata and can be supplied to deployment tooling.

If validation fails or the profile revision changed between read and export, stop the handoff and refresh the profile state.

Validate candidate Firefox policies.json through the API

Use the validation endpoint as a preflight gate for candidate Firefox Enterprise `policies.json` documents and structured error handling.

Choose the target schema channel, such as `release-153` or `esr-140.13`, and prepare a `ValidationRequest` body with a full Firefox document in `document.policies`.

This task covers `API-VAL-001`. BPM validates candidate `policies.json` content before import, export handoff, or an external policy-review gate. A plain policy mapping is still accepted for compatibility, but new integration examples use the full Firefox `policies.json` shape.

1. Submit a successful validation request for the selected schema channel.

   Request example: `POST $BPM_BASE_URL/api/validate/release-153` with JSON `{"document":{"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}}`. Response example: `200 {"ok":true,"profile":"release-153"}`.

2. Treat `ok=false` as a failed gate even when HTTP status is `200`.

   Request example: `POST $BPM_BASE_URL/api/validate/release-153` with JSON `{"document":123}`. Response example: `200 {"ok":false,"profile":"release-153","detail":"Expected object with policy mappings","error":"Expected object with policy mappings"}`. Check the response body, not HTTP status alone.

3. Handle malformed canonical Firefox document errors as request-shape failures.

   Request example: `POST $BPM_BASE_URL/api/validate/release-153` with JSON `{"document":{"policies":[]}}`. Error example: `400` with `detail.message="Firefox policies.json validation failed"` and `detail.issues[].path` pointing to `policies`.

4. Reject unsupported schema channels before trusting validation results.

   Request example: `POST $BPM_BASE_URL/api/validate/beta-999` with JSON `{"document":{"policies":{"DisableTelemetry":true}}}`. Error example: `404` with detail text `Unknown profile 'beta-999'`. A registered but unavailable schema returns `503`.

5. Map policy validation failures to actionable reviewer feedback.

   Request example: `POST $BPM_BASE_URL/api/validate/release-153` with JSON `{"document":{"policies":{"Proxy":{"Mode":"bogus"}}}}`. Error example: `422` with `detail.message="Policy validation failed"` and issue fields `policy`, `path`, and `message`.

6. Record the validation decision in the integrating product.

> Warning:
> Validation is a schema and policy-shape gate only. It does not deploy Firefox, prove runtime enforcement, authenticate callers, reserve a profile revision, or provide an idempotency key. Store the channel, candidate document hash, response status, `ok` value, structured issues, and reviewer decision externally.

The integration can block unsafe candidate `policies.json` handoffs and preserve structured evidence for success and failure decisions.

If validation succeeds, continue to import, profile update, or export handoff using the same schema channel and recorded candidate hash.

Run lifecycle workflows and recover from failures

Run a pull, compare, and update control-product scenario

Use existing profile and validation APIs to pull BPM state, compare it externally, and update one profile with explicit ownership and concurrency evidence.

Set `$BPM_BASE_URL` and decide which external control product owns comparison logic, desired-state calculation, retry policy, and audit records. BPM does not expose a compare API or transaction wrapper.

This scenario uses `API-PROFILE-001`, `API-PROFILE-003`, `API-PROFILE-005`, and `API-VAL-001`. BPM owns stored profile state and validation. The integrating product owns desired state, comparison output, reviewer approval, and retry decisions.

1. Pull candidate profiles from the Library with the same filter rules the control product uses for ownership.

   Request example: `GET $BPM_BASE_URL/api/profiles?q=corp&lifecycle=active&limit=50&offset=0`. Response example: `200 ProfileRead[]`. Error example: `422` for invalid query input.

2. Read each selected profile by ID and store its current `revision`.

   Request example: `GET $BPM_BASE_URL/api/profiles/42`. Response example: `200 ProfileRead` with `flags`, `compliance`, `schema_version`, and `revision`. Error examples: `404` missing or archived profile; `422` invalid path input.

3. Compare the pulled profile with external desired state outside BPM.

   Expected result: the integrating product records changed policy keys, unchanged keys, missing keys, reviewer, and source of desired state. Do not assume a BPM compare endpoint, marketplace connector, or certified workflow exists.

4. Validate the candidate Firefox document before applying the update.

   Request example: `POST $BPM_BASE_URL/api/validate/release-153` with JSON `{"document":{"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}}`. Response example: `200 {"ok":true,"profile":"release-153"}`. Error examples: `400`, `404`, `422`, or `503` depending on document shape, channel, policy validity, and schema availability.

5. Patch the BPM profile only with the revision observed before comparison.

   Request example: `PATCH $BPM_BASE_URL/api/profiles/42` with JSON `{"flags":{"DisableTelemetry":true,"BlockAboutConfig":true},"expected_revision":7,"compliance":{"source":"control-product","decision":"approved"}}`. Response example: `200 ProfileRead` with incremented `revision`. Error examples: `400`, `404`, `409` stale `expected_revision`, or `422`.

6. Handle failures without automatic replay.

> Warning:
> If validation fails, the revision is stale, or the update result does not match the intended desired state, stop the workflow, re-read BPM state, and create an external audit record. BPM does not provide a multi-operation transaction, automatic rollback, idempotency key, or safe retry contract.

The control product has an executable pull/compare/update flow with explicit state ownership, validation evidence, revision protection, and failure recovery boundaries.

If the updated profile must be deployed elsewhere, continue with the Firefox export task and preserve the profile ID, revision, validation response, and approval record.

Run an import, review, export, and compliance handoff scenario

Use existing health, import, validation, profile, and export APIs to hand off a reviewed Firefox document with compliance evidence.

Set `$BPM_BASE_URL`, choose the schema channel, and decide which system owns the source document, compliance scan result, reviewer approval, exported artifact, and recovery decision.

This scenario uses `API-HEALTH-001`, `API-HEALTH-002`, `API-FF-001`, `API-PROFILE-003`, `API-VAL-001`, and `API-FF-002`. BPM stores the normalized profile and opaque compliance metadata; the integrating product owns scan interpretation and downstream deployment.

1. Start with health and readiness before the handoff sequence.

   Request examples: `GET $BPM_BASE_URL/health` and `GET $BPM_BASE_URL/health/ready`. Response examples: `200 {"status":"ok"}` and `200 {"status":"ready","ready":true}`. Error example: transport or gateway failure is handled outside BPM because no application error is currently declared.

2. Import the reviewed Firefox document with compliance metadata attached to the BPM profile.

   Request example: `POST $BPM_BASE_URL/api/profiles/import/firefox/policies.json` with JSON `{"name":"reviewed-corp-baseline","schema_version":"release-153","compliance":{"scanner":"external-control","result":"passed","ticket":"SEC-42"},"document":{"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}}`. Response example: `201 ProfileRead`. Error examples: `400`, `409`, `415`, or `422`.

3. Read the created profile and record BPM ownership fields separately from external evidence.

   Request example: `GET $BPM_BASE_URL/api/profiles/42`. Response example: `200 ProfileRead` with `id`, `revision`, `flags`, `schema_version`, and opaque `compliance`. Error examples: `404` or `422`.

4. Validate the canonical Firefox document that will be handed off.

   Request example: `POST $BPM_BASE_URL/api/validate/release-153` with JSON `{"document":{"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}}`. Response example: `200 {"ok":true,"profile":"release-153"}`. Error examples: `400`, `404`, `422`, or `503`.

5. Export the BPM profile as the canonical Firefox deployment document.

   Request example: `GET $BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?download=1&pretty=1`. Response example: `200 application/json` with `Content-Disposition` and top-level `policies`. BPM profile metadata and compliance metadata are not exported.

6. Stop on mismatches and write the failure recovery record.

> Warning:
> If import, validation, read-back, or export evidence differs from the approved source document, do not retry blindly. Store source hash, profile ID, revision, validation body, export hash, scanner result, reviewer, response status, and the recovery decision outside BPM.

The integrating product has an executable import-review-export flow with compliance handoff, canonical export, and explicit recovery evidence.

Use deployment tooling outside BPM to apply the exported `policies.json`; BPM validation does not prove Firefox runtime enforcement.

Pull and read BPM profiles from an external control product

Build the pull/list/read side of a control-product integration with explicit ownership, filters, and evidence boundaries.

Set `$BPM_BASE_URL`. Decide which external control product owns inventory selection, comparison storage, retry policy, and audit evidence; BPM owns only the profile records returned by its API.

This workflow uses `API-PROFILE-001`, `API-PROFILE-002`, and `API-PROFILE-003`. It is the safe read-only entry point before compare/update, validate-before-apply, import-review-export, or compliance metadata handoff workflows.

1. Confirm the control-product scope before reading BPM state.

   Record the external owner, purpose, schema channel, filter rules, and whether archived profiles are excluded. Use `lifecycle=active` unless a recovery procedure explicitly needs `include_deleted=true`.

2. List profiles with deterministic filters and paging.

   Request example: `GET $BPM_BASE_URL/api/profiles?q=corp&lifecycle=active&limit=50&offset=0&sort=updated_at&order=desc`. Response example: `200 ProfileRead[]`. Error example: `422` for invalid query parameters.

3. Capture optional inventory counts separately from profile data.

   Request example: `GET $BPM_BASE_URL/api/profiles/stats?q=corp&lifecycle=active`. Response example: `200 application/json`. Store count evidence as advisory inventory context, not as a lock or transaction boundary.

4. Read each selected profile by ID before comparison.

   Request example: `GET $BPM_BASE_URL/api/profiles/42?include_deleted=false`. Response example: `200 ProfileRead` with `id`, `revision`, `schema_version`, `flags`, and `compliance`. Error examples: `404` or `422`.

5. Persist a read snapshot outside BPM for the control product.

   Store profile ID, revision, schema channel, filter query, read timestamp, source URL, selected policy keys, and the control-product run ID. Do not store secrets or private host names in shared documentation examples.

6. Stop if the pull result cannot be trusted.

> Warning:
> If paging, filtering, or read-back evidence is incomplete, stop before update. BPM does not expose cursor pagination, snapshot isolation, compare locks, webhooks, or a safe replay contract for read workflows.

The external control product has a reproducible pull/list/read snapshot and knows which evidence belongs to BPM and which evidence belongs to the external system.

Continue with validate-before-apply or compare/update only after the external system has recorded the selected profile IDs and revisions.

Validate and apply a profile update with expected_revision

Run a validation-first update workflow that protects BPM state with the current revision and stops on stale or invalid data.

The external product already read the target profile and stored its current `revision`. Prepare the candidate Firefox `policies.json` document before sending any profile patch.

This workflow uses `API-VAL-001`, `API-PROFILE-003`, and `API-PROFILE-005`. It documents the current optimistic update path; BPM does not provide ETags, precondition headers, idempotency keys, or multi-operation transactions.

1. Build the candidate Firefox document outside BPM.

   Record desired policy keys, source system, reviewer, ticket, and schema channel. Example candidate: `{"document":{"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}}`.

2. Validate the candidate before patching the profile.

   Request example: `POST $BPM_BASE_URL/api/validate/release-153`. Response example: `200 {"ok":true,"profile":"release-153"}`. Error examples: `400`, `404`, `422`, or `503`.

3. Re-read the profile immediately before applying the update when the approval window is long.

   Request example: `GET $BPM_BASE_URL/api/profiles/42`. Response example: `200 ProfileRead`. If `revision` changed, re-run comparison and approval rather than patching stale state.

4. Patch only mutable fields with the observed revision.

   Request example: `PATCH $BPM_BASE_URL/api/profiles/42` with JSON `{"flags":{"DisableTelemetry":true,"BlockAboutConfig":true},"expected_revision":7,"compliance":{"source":"control-product","decision":"approved"}}`. Response example: `200 ProfileRead` with incremented `revision`.

5. Treat conflict responses as stop signals.

   Error examples: `400` invalid payload, `404` missing profile, `409` stale `expected_revision`, and `422` schema-level request errors. A `409` means the external desired state must be recalculated.

6. Record update evidence and avoid blind replay.

   > Warning:
   > Do not retry the same PATCH blindly. Store old revision, new revision, validation body, patch body hash, response status, reviewer approval, and the retry decision outside BPM.

The profile update is applied only after validation and revision checks, with explicit audit evidence for success or conflict.

Export the profile or hand off compliance metadata only after the read-back profile matches the approved desired state.

Run import, review, export, and compliance metadata handoff

Connect an external review or compliance product to BPM import and export APIs without treating BPM metadata as Firefox deployment content.

Set `$BPM_BASE_URL` and prepare a reviewed Firefox `policies.json` source document plus external compliance metadata such as scanner, ticket, reviewer, and decision.

This workflow uses `API-FF-001`, `API-PROFILE-003`, `API-VAL-001`, and `API-FF-002`. BPM stores opaque compliance metadata on the profile, but exported Firefox JSON contains only deployable `policies`.

1. Import the reviewed source document into a BPM profile.

   Request example: `POST $BPM_BASE_URL/api/profiles/import/firefox/policies.json` with JSON `{"name":"reviewed-corp-baseline","schema_version":"release-153","compliance":{"scanner":"external-control","ticket":"SEC-42"},"document":{"policies":{"DisableTelemetry":true}}}`. Response example: `201 ProfileRead`.

2. Read back the created profile and separate BPM fields from external evidence.

   Request example: `GET $BPM_BASE_URL/api/profiles/42`. Response example: `200 ProfileRead` with `revision`, `flags`, and opaque `compliance`. Error examples: `404` or `422`.

3. Review the normalized policy set before export.

   The reviewer checks imported keys, schema channel, source document hash, compliance metadata, and any BPM normalization warnings. BPM does not certify the external scanner result or Firefox runtime enforcement.

4. Validate the deployment document that will be exported.

   Request example: `POST $BPM_BASE_URL/api/validate/release-153` with JSON `{"document":{"policies":{"DisableTelemetry":true}}}`. Response example: `200 {"ok":true}`. Error examples: `400`, `404`, `422`, or `503`.

5. Export the canonical Firefox policies document.

   Request example: `GET $BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?download=1&pretty=1`. Response example: `200 application/json` with `Content-Disposition` and top-level `policies`. Compliance metadata are not exported.

6. Write the handoff record before downstream deployment.

   > Warning:
   > Store source hash, profile ID, revision, validation response, export hash, scanner result, reviewer, and downstream owner outside BPM. Do not treat BPM export as proof that Firefox has applied the policy.

The external product receives a canonical Firefox deployment document and a separate compliance handoff record with clear ownership boundaries.

Use deployment tooling outside BPM to place the exported `policies.json` on managed Firefox systems.

Gate control-product startup with BPM health and readiness

Use the current health probes as a startup gate before an external control product begins API synchronization.

The BPM source deployment is already started. Configure the external product with `$BPM_BASE_URL`, timeout, polling interval, maximum wait, and the action to take when the gate fails.

This workflow uses `API-HEALTH-001` and `API-HEALTH-002`. The probes are small preflight signals only; they do not expose authentication, authorization, database diagnostics, schema-cache diagnostics, request IDs, or service-level objectives. Keep health-gated startup, data ownership, failure recovery, and retry/backoff decisions in the external control product.

Documentation-assistant availability is separate from core BPM health and readiness. In 0.9.3, every assistant question receives the published local-model training notice; it does not start a model, retrieval, RAG index, or external-source request. This boundary does not block deterministic documentation search and must not gate profile mutations or API synchronization.

1. Check basic liveness before opening the synchronization queue.

   Request example: `GET $BPM_BASE_URL/health`. Response example: `200 {"status":"ok"}`. Transport errors are handled by the external product because no BPM application error body is declared for this probe.

2. Check readiness before profile or validation calls.

   Request example: `GET $BPM_BASE_URL/health/ready`. Response example: `200 {"status":"ready","ready":true}`. If `ready` is not true, hold the queue and do not start mutations.

3. Record the startup gate decision.

   Store timestamp, base URL label, timeout, probe status, observed body, control-product version, and chosen next action. Do not store private hosts in shared runbook examples.

4. Start with read-only inventory after the gate passes.

   Call `GET $BPM_BASE_URL/api/profiles?limit=50` before import, export, validation, or patch operations. This keeps initial synchronization observable and recoverable.

5. Move to mutation workflows only after read-only calls succeed.

   Continue to validate-before-apply, import-review-export, or export workflows according to the external product schedule. Keep retry and backoff policy outside BPM.

6. Fail closed when the gate is ambiguous.

> Warning:
> A successful probe does not reserve revisions or guarantee later API success. A failed or ambiguous probe must stop automated mutations until an operator checks logs, source deployment state, and network/CORS boundaries.

The external control product starts synchronization only when BPM responds to the current health and readiness handshake.

If the startup gate fails, use the Administrator/DevOps troubleshooting and source-deployment topics rather than adding undocumented probe assumptions.

Recover from integration failures and record audit evidence

Handle validation, import/export, readiness, and revision failures without unsafe retries or hidden transaction assumptions.

Use this workflow when an external product receives a failed BPM API response, timeout, unexpected response body, stale `expected_revision`, or mismatched export evidence.

This workflow connects `API-HEALTH-001`, `API-HEALTH-002`, `API-PROFILE-003`, `API-PROFILE-005`, `API-FF-001`, `API-FF-002`, and `API-VAL-001`. It defines operator evidence; BPM does not provide distributed tracing, automatic rollback, bulk transaction recovery, or webhook notifications.

1. Classify the failure before retrying.

   Record whether the failure is health/readiness, validation, import, export, read, patch, conflict, timeout, or transport. Preserve HTTP status such as `400`, `404`, `409`, `415`, `422`, or `503`.

2. Freeze the external desired state and BPM observation.

   Store request method, path, sanitized payload hash, profile ID, observed `revision`, schema channel, response body, response time, and control-product run ID. Never include secrets or private host names in shared evidence.

3. Re-check BPM health and re-read the profile when the failure is recoverable.

   Request examples: `GET $BPM_BASE_URL/health`, `GET $BPM_BASE_URL/health/ready`, and `GET $BPM_BASE_URL/api/profiles/42`. If the profile revision changed, return to comparison and approval.

4. Validate or export again only after an operator-approved decision.

   For validation, repeat `POST $BPM_BASE_URL/api/validate/release-153` with the same candidate document. For export, repeat `GET $BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?pretty=1` and compare the exported document hash.

5. Stop mutation retries on stale revision or ambiguous evidence.

   If `expected_revision` returns `409`, if export hash differs, or if validation errors changed, do not continue the old run. Open a new external run with fresh read, comparison, and reviewer approval.

6. Publish the audit record to the external system of record.

> Warning:
> BPM stores profile state but does not store a complete integration audit trail. The external product must record owner, decision, ticket, retry count, stop reason, old and new revisions, hashes, and operator notes.

Failures are recoverable only through explicit evidence, fresh reads, and operator-approved retry decisions, not through blind replay.

After recovery, link the audit record to the successful profile revision or mark the integration run as stopped.

Troubleshoot BPM and assess production readiness

Troubleshoot failed startup and unreachable probes

Diagnose a source-run BPM process that does not start or does not answer health and readiness probes.

Use this procedure when `make dev`, `uvicorn app.main:app --reload --port 8000`, `GET /health`, or `GET /health/ready` fails during source deployment or integration startup.

This diagnostic keeps process, environment, and probe evidence separate. It does not create a systemd unit, Windows service, reverse proxy, TLS endpoint, or production restart policy.

1. Capture the failing command and foreground log.

   Run or copy the exact command, then record stdout and stderr from the foreground terminal. Include `make dev`, `uvicorn app.main:app --reload --port 8000`, Python traceback summary, working directory, and source revision from `git rev-parse --short HEAD`.

2. Verify the configured host and port before changing the deployment.

   Check `BPM_HOST`, `BPM_PORT`, `BPM_RELOAD`, `BPM_DATABASE_URL`, and `BPM_DOCUMENTATION_SITE_DIR`. Recheck `http://127.0.0.1:8000/health` and `http://127.0.0.1:8000/health/ready` after correcting a configuration value.

3. Separate process startup from probe reachability.

   If the process is not running, fix dependency, environment, or migration evidence first. If the process is running but probes are unreachable, check local firewall, bind address, WSL port forwarding, and whether the request targets `127.0.0.1`, `0.0.0.0`, or a Windows host route.

4. Preserve user data while collecting evidence.

   Copy logs, environment variable names, and probe responses only. Do not edit the database, schema cache, or deployment files as routine recovery. Do not delete `data/bpm.db` to make startup pass.

5. Stop dependent integrations and escalate when probes remain unreachable.

   > Warning:
   > If probes remain unreachable after configuration and process checks, stop automated integration runs and escalate to the BPM deployment owner with the command, logs, configuration names, and probe evidence.

The operator can distinguish a failed process, an unreachable probe, and a deployment boundary without mutating user data.

After probes pass, return to the health-gated startup or reusable API examples topic before running mutating integrations.

Troubleshoot schema cache and API validation errors

Diagnose unavailable schema cache, unknown schema channel, and Firefox policies validation failures without patching generated schema artifacts.

Use this procedure when `POST $BPM_BASE_URL/api/validate/release-153`, `POST $BPM_BASE_URL/api/validate/beta-999`, or user-facing validation returns `400`, `404`, `422`, or `503`; a non-object document can return `200 ok=false` with `Expected object with policy mappings`.

BPM validation depends on maintained schema channels and local schema/cache configuration. The diagnostic records whether the issue is request shape, unknown profile, invalid Firefox policy data, or unavailable schema cache.

1. Record the exact validation request and response.

   Store sanitized request body hash, schema channel, HTTP status, `detail`, `message`, `error`, and `issues`. Keep the candidate `policies.json` outside logs if it contains customer values.

2. Classify request-shape failures first.

   A non-object `document` or missing `document.policies` is operator input, not a schema-cache defect; current BPM can report this as `200 ok=false`. Focused rerun: `POST $BPM_BASE_URL/api/validate/release-153` with `{"document":{"policies":{"DisableTelemetry":true}}}`.

3. Classify schema-channel and cache failures.

   A profile such as `beta-999` can return `404`; an unavailable registered schema can return `503`. Check `BPM_SCHEMA_BASE_URL`, `BPM_SCHEMA_CACHE_DIR`, and `BPM_SCHEMA_HTTP_TIMEOUT` before any broader rebuild.

4. Use the current schema recovery boundary.

   Do not hand-edit the schema cache or generated Firefox policy references. If the cache remains unavailable after the documented configuration checks, preserve the request and environment evidence and escalate to the BPM deployment owner.

5. Stop unsafe recovery and preserve evidence.

   If the same valid request changes status across runs, preserve response bodies, source revision, cache path, and environment variables.

   > Warning:
   > Do not edit generated schema artifacts or database rows as routine recovery; separate product defects from operator payload errors.

The operator has a classified validation failure with a smallest rerun and no hidden schema mutation.

After validation succeeds, return to validate-before-apply or import-review-export workflows.

Troubleshoot Firefox import and export failures

Diagnose JSON import, multipart import, and Firefox policies export failures while preserving source and exported evidence.

Use this procedure when `POST $BPM_BASE_URL/api/profiles/import/firefox/policies.json` or `GET $BPM_BASE_URL/api/export/profiles/42/firefox/policies.json` returns `400`, `404`, `409`, `415`, or `422`.

Import/export troubleshooting separates source document validity, multipart field shape, duplicate profile names, archived profile access, and canonical Firefox export from downstream deployment tooling.

1. Identify the operation and content type.

   Record whether the request was JSON `FirefoxPoliciesJsonImportRequest`, `multipart/form-data` with `file=@policies.json`, or export with `download=1`, `pretty=1`, or `include_deleted=true`.

2. Validate the source Firefox document before retrying import.

   Focused rerun: `POST $BPM_BASE_URL/api/validate/release-153` with the same `document.policies`. If validation fails, fix the source document outside BPM before another import.

3. Check import-specific operator errors.

   For `409`, choose a new synthetic profile name or archive/restore intentionally. For `415`, verify `application/json` or multipart upload fields. For `422`, record field-level errors and do not rewrite profile storage directly.

4. Check export-specific state.

   For `404`, re-read `GET $BPM_BASE_URL/api/profiles/42?include_deleted=true`. For archived profiles, decide whether `include_deleted=true` is appropriate. Compare exported document hash with the reviewed source hash; BPM profile metadata and compliance metadata are not exported.

5. Record recovery and stop conditions.

   Keep the request and response evidence with the external control-product record before retrying a corrected operation.

> Warning:
> Do not retry blindly after mismatched import/export evidence; store source hash, export hash, profile ID, revision, HTTP status, and operator decision.

The operator can identify whether the failure belongs to source JSON, multipart fields, profile lifecycle state, or export evidence.

After recovery, attach the evidence to the external control-product audit record.

Troubleshoot database and storage issues

Diagnose local SQLite, storage path, backup, and profile visibility issues without editing database rows directly.

Use this procedure when profiles disappear, writes fail, migration evidence is unclear, backup checksums do not match, or `BPM_DATABASE_URL` points to an unexpected location.

Current source deployment uses local storage such as `sqlite+aiosqlite:///./data/bpm.db` and repository-owned files under `data/`, schema cache paths, and documentation site paths. There is no managed backup or restore orchestration in BPM.

1. Confirm the active database URL and working directory.

   Record `BPM_DATABASE_URL`, current directory, source revision, and whether the process runs from Linux filesystem or `/mnt/c`. Do not assume two shells point at the same `data/bpm.db`.

2. Check migration and profile API evidence before opening storage files.

   Focused rerun: `alembic upgrade head`, then `curl -fsS http://127.0.0.1:8000/api/profiles?limit=50`. If the API returns expected profiles, the symptom may be filter, lifecycle, or browser state.

3. Verify backup evidence before destructive recovery.

   Check `mkdir -p backups`, `cp data/bpm.db backups/bpm-pre-update.db`, and `sha256sum backups/bpm-pre-update.db` records. Keep exports of critical `policies.json` profiles as separate evidence.

4. Avoid direct database row edits.

   Do not edit SQLite rows, generated artifacts, or schema cache files as routine recovery. Use documented profile archive, restore, import, export, or update-from-source procedures instead.

5. Escalate with storage evidence when recovery is unsafe.

   Include database URL, file path, checksum, migration command, API response, and backup/export evidence.

   > Warning:
   > If no verified pre-failure backup exists, stop before deleting or replacing `data/bpm.db`; BPM does not provide point-in-time recovery.

The operator has a storage diagnosis that preserves user data and separates database state from UI or filter symptoms.

After storage is stable, rerun health/readiness and the smallest affected API workflow.

Troubleshoot WSL networking and stale dependencies

Diagnose Windows 10/11 through WSL access problems, Linux filesystem placement, and stale Python dependencies after source changes.

Use this procedure when Windows cannot reach a WSL-hosted BPM process, probes work inside WSL but not from Windows, file performance is unstable under `/mnt/c`, or dependencies appear stale after `git checkout <approved-new-ref>`.

WSL troubleshooting keeps Windows browser access, Linux process binding, filesystem placement, and Python environment refresh separate. BPM does not provide a native Windows service or installer in 0.9.1.

1. Confirm where the source checkout and virtual environment live.

   Prefer the Linux filesystem for the checkout and `.venv`. Record whether files are under `/mnt/c`, which can change performance and permissions. Do not mix multiple checkouts during diagnosis.

2. Verify the WSL process binding and Windows route.

   Check `BPM_HOST="0.0.0.0"`, `BPM_PORT="8000"`, `curl -fsS http://127.0.0.1:8000/health` inside WSL, and browser access from Windows. If Windows fails, record firewall and port-forwarding evidence.

3. Refresh dependencies after source movement or revision changes.

   Focused rerun: `source .venv/bin/activate`, `python -m pip install --upgrade pip`, and `pip install -e ".[dev]"`. If dependency errors persist, record package names and source revision rather than editing installed packages manually.

4. Separate stale dependency symptoms from schema or database problems.

   If import/export or validation fails after dependency refresh, use schema/cache and import/export troubleshooting. If profile visibility changes, use database/storage troubleshooting.

5. Stop unsafe native-Windows assumptions.

   Do not create undocumented Windows services, move the database between filesystems, or claim native installer behavior.

   > Warning:
   > If WSL networking remains ambiguous, stop external integrations and escalate with WSL distro, checkout path, bind address, Windows URL, and probe outputs.

The operator can distinguish WSL routing, filesystem placement, and dependency-refresh defects without changing product storage.

After WSL access is stable, return to source deployment verification and health-gated startup.

Troubleshoot an unavailable documentation portal

Collect the visible /help/ state and deployment evidence when the installed documentation portal is unavailable.

Use this procedure when an installed BPM instance reports a missing, stale, incompatible, incomplete, or not-found state at `/help/`.

The documentation portal is part of the BPM interface. Its availability depends on the installed documentation artifact matching the running BPM version; BPM does not repair a missing or incompatible artifact at runtime.

1. Record the portal URL and visible state.

   Capture the full `/help/` URL, locale, browser message, BPM version, and time of the failure. Keep the response separate from FastAPI `/docs` and `/openapi.json`, which are different interfaces.

2. Confirm the running BPM instance and its documentation setting.

   Record the process host, port, source revision or deployed release identifier, and `BPM_DOCUMENTATION_SITE_DIR` value without copying secrets.

3. Stop dependent documentation actions when the state is not ready.

   > Warning:
   > Do not treat a missing or incompatible portal as a working help system, and do not hand-edit installed files to bypass the state.

4. Escalate the evidence to the BPM deployment owner.

   Provide the portal URL, visible state, BPM version, documentation-site setting name, process logs, and the time of the observation. The deployment owner can restore a matching documentation artifact through the maintained documentation procedure.

5. Verify the portal after recovery.

   After the deployment owner restores a matching artifact, reopen `/help/` and the affected contextual help routes.

The operator has enough evidence to distinguish an unavailable documentation artifact from the BPM API and to request the correct recovery.

Record the recovered portal state with the original escalation evidence.

Review network exposure and proxy-readiness questions

Prepare network exposure notes and reverse-proxy questions without publishing unsupported TLS or proxy recipes as BPM behavior.

The current single-node source readiness assessment is complete, and the operator knows whether users will reach BPM through loopback, a WSL forwarded address, a lab host, or an externally owned proxy.

BPM runs on a configured host and port. Reverse proxy, TLS, authentication gateway, header policy, and rate-limiting decisions belong to the operating organization.

1. Choose the smallest network exposure that satisfies the current evaluation.

   Prefer `127.0.0.1`. Use `BPM_HOST="0.0.0.0"` only when WSL forwarding or a controlled lab network requires it, and record why `BPM_PORT="8000"` is reachable.

2. Write down proxy-readiness questions before adding an external proxy.

   Answer who owns TLS certificates, hostname routing, request size limits, timeout values, CORS origins through `BPM_CORS_ALLOW_ORIGINS`, forwarded headers, access logs, and incident response.

3. Verify that probes still work through the chosen boundary.

   ```
   curl -fsS http://127.0.0.1:8000/health
   curl -fsS http://127.0.0.1:8000/health/ready
   ```

   If an externally owned proxy is used in a lab, record both direct source probes and proxy-side probes. Do not treat a proxy 200 response as proof of BPM authentication, authorization, or HA.

4. Record platform-managed network controls.

   > Warning:
   > Document the owner and operating procedure for proxying, TLS, WAF rules, request signing, rate limiting, SSO, OAuth/OIDC, role-based access control, and production hardening.

The network note records the chosen exposure and the organization-owned controls around it.

Use the monitoring, backup, and update-window topic before a shared evaluation window starts.

Plan monitoring inputs, backup evidence, and update windows

Prepare operational evidence for monitoring, backup, and source update windows.

The instance has passed source readiness and network exposure review, and a single administrator or DevOps owner is responsible for the evaluation window.

Operations planning for a source-run instance uses observable inputs, backup and export evidence, and explicit update windows.

1. List the monitoring inputs that exist today.

   Record process status, foreground terminal output, `stdout`, `stderr`, `GET /health`, `GET /health/ready`, selected API example results, documentation access through `/help/`, and the source revision from `git rev-parse --short HEAD`.

2. Collect backup and export evidence before the window.

   ```
   mkdir -p backups
   cp data/bpm.db backups/bpm-pre-update.db
   sha256sum backups/bpm-pre-update.db
   ```

   Also export representative Firefox documents such as `GET $BPM_BASE_URL/api/export/profiles/42/firefox/policies.json` and store the resulting `policies.json` evidence with checksums.

3. Schedule a source update window.

   Plan downtime before refreshing the approved source revision and dependencies, applying `alembic upgrade head`, and completing the required runtime verification.

4. Record platform-owned controls.

   > Warning:
   > Document the operating procedures for monitoring, backup scheduling, restore, recovery, secrets, token rotation, and update orchestration. Use the evidence and stop conditions from the update runbook.

The operation plan contains monitoring inputs, backup/export evidence, and a realistic update window for current source operation.

Attach this evidence to the production operating record.

BPM documentation assistant

Operate the documentation assistant in BPM 0.9.3

Verify the released training-notice boundary without making BPM or deterministic documentation search dependent on the chat.

Use a supported BPM 0.9.3 source checkout and the base system capacity in Minimum system requirements for BPM.

In this release the chat is a localized reader interface only. It does not start or install a model, retrieve documentation, use an E5 index, generate an answer, expose citations, or contact external sources. Deterministic documentation search remains separate and available.

1. Start BPM by using the normal supported source workflow.

   Do not add model, index, embedding, or external-provider setup to BPM startup for release 0.9.3.

2. Open the documentation portal in the reader's locale.

   Confirm that the assistant can be opened from the lower-right corner without changing the ordinary search controls.

3. Submit a BPM-related question.

   The transcript must show the localized notice that local-model training is in progress and will be available in a later version.

4. Submit an unrelated question.

   The same notice is expected; the chat is not a general-purpose assistant in this release.

5. Use deterministic documentation search to find the current answer.

   Search, filters, navigation, and result ordering remain the supported product-information path.

The assistant presents the released localized training notice, while BPM and deterministic documentation search remain independently usable.

Do not install a model or build a RAG generation for 0.9.3. Record a documentation defect if the notice, locale, or ordinary search boundary differs from this behavior.

Maintain the documentation-assistant boundary in BPM 0.9.3

Keep the released localized training notice and deterministic-search boundary accurate while maintaining BPM documentation.

Have the approved BPM source revision and documentation publication workflow available.

There is no local model, model artifact, RAG generation, embedding store, citation store, or external-source provider to maintain in 0.9.3. Do not create maintenance state for features that this release does not deliver.

1. Apply the approved BPM source revision and rebuild the documentation.

   Use the normal documentation workflow, including `make docs-install-dev` when preparing a development checkout.

2. Open the published documentation in every supported locale.

   Confirm that the assistant entry remains available and that deterministic search still works independently.

3. Submit one question in each locale.

   Verify the locale-owned training notice and confirm that no answer, source, citation, external claim, or model-installation flow is presented.

4. Do not run model-installation or RAG-generation commands for 0.9.3.

   They belong to a later product release and are not a recovery path for the training notice.

5. Record the documentation revision and verification result.

   Escalate a mismatch between the released notice and the deterministic-search boundary as a documentation defect.

The published portal preserves a localized training notice for the chat and a separate current-information path through deterministic search.

Do not retain user questions as a maintenance artifact. Reassess model and retrieval operations only when a later release explicitly delivers them.